

Projet PCII - Saclay Valley

Rapport de Projet

Juralaine LI Andrei-Vladimir MUNGIU
Lorie QIU Raphaël ARAKELIAN

29 avril 2026



Figure 1 – Bienvenue à Saclay Valley !

Table des matières

1	Introduction	4
2	Analyse	4
2.1	Analyse Globale	4
2.2	Analyse Détaillée	5
3	Plan de Développement	9
3.1	Répartition et charge de travail	10
4	Conception Générale	18
4.1	Architecture Globale	18
4.2	Schéma de l'Architecture Fonctionnelle	18

4.3	Description Détaillée des Blocs Fonctionnels	19
5	Conception détaillée	20
5.1	Fonctionnalité 1 : Terrain et Grille	20
5.1.1	Création et initialisation de la grille : Terrain et objets initiaux . . .	21
5.1.2	Gestion des différents types de cases	21
5.1.3	Création de la vue principale : Affichage du terrain	22
5.1.4	Scrolling de la vue	23
5.2	Gestion des jardiniers	26
5.2.1	Le Cycle de Vie et la Programmation Concurrente	26
5.2.2	Système d'Actions et Logique Métier	27
5.2.3	Le Système d'Inventaire : Gestion et Affichage des Ressources . . .	34
5.2.4	Gestion Concurrente de Plusieurs Jardiniers	36
5.3	Cycle Végétal & Eau	39
5.3.1	Cycle de Vie des Plantes	39
5.3.2	Consommation d'eau et Survie : Gestion de l'Humidité	40
5.3.3	Interactions Agricoles : Lien entre Grille et Logique Biologique . . .	42
5.4	Entités et Déplacements (Pathfinding)	45
5.4.1	Algorithme de Recherche de Chemin (A*)	45
5.4.2	Gestion des Ennemis et Nuisibles : Entités Hostiles et Concurrence	46
5.5	Constructions et Bâtiments	48
5.5.1	Le BuildingManager : Contrôle et Placement	49
5.5.2	Catalogue des Bâtiments et Infrastructures : Fonctionnalités et Effets	50
5.6	Économie et Progression	52
5.6.1	L'Expérience et le Passage de Niveaux	52
5.6.2	Le Système de Quêtes et de Chapitres	54
5.6.3	Le Déblocage des Fonctionnalités	55
5.6.4	Le Système d'Économie :	58
5.7	Interface et Contrôles	60
5.7.1	Popups Informatifs et Système de Menus	61
5.7.2	Barre d'Action Rapide (Hotbar) et Équipements	62
5.7.3	Interaction Spatiale et Retours Visuels	64
5.8	Menu et Sauvegarde	66
5.8.1	Écran d'Accueil et Navigation	66
5.8.2	Architecture de la Sauvegarde	67
5.8.3	Processus de Sérialisation et Flux d'Entrée/Sortie (SaveManager) .	69
5.8.4	Mécanisme de Pause et Suspension d'Exécution	72
6	Résultat : Saclay Valley	76
6.1	Aperçu Global et Expérience Utilisateur	76
6.2	Comportement Systémique et Fluidité	76
6.3	Une Boucle de Jeu Aboutie	77

7	Documentation Utilisateur	78
7.1	Prérequis Techniques et Lancement	78
7.2	Premier Démarrage et Gestion des Parties	78
7.3	Contrôles et Navigation Visuelle	78
7.4	De la Terre à la Vente	78
7.5	Construire et Gérer les Menaces	79
7.6	Mettre le Jeu en Pause et Quitter	79
8	Documentation Développeur : Guide de Reprise et d'Évolution	80
8.1	Point d'Entrée et Classes Fondamentales	80
8.2	Principales Constantes et Équilibrage	80
8.3	Pistes d'Amélioration et Implémentations Futures	81
8.3.1	Système de Lootbox	81
8.3.2	Mécanique de Compostage et Gestion des Déchets	81
9	Conclusion et Perspectives	82

1 Introduction

Ce projet, réalisé dans le cadre de l'unité d'enseignement de Programmation Concurrente et Interface Interactive (PC2I), a pour objectif de concevoir et développer une simulation de gestion agricole et de jardinage en Java en langage Java. L'application met le joueur aux commandes d'un terrain aménageable, représenté sous la forme d'une grille, où il doit gérer le développement d'une exploitation prospère. Le principe du jeu repose sur la gestion de ressources afin de faire grandir une ferme. Le joueur doit interagir avec son environnement via une interface graphique interactive. Il doit assigner des tâches spécifiques à ses jardiniers, telles que le labourage, la plantation, et la récolte, cela tout en gérant ses ressources, eau, argent, graines, etc. Le système intègre une mécanique temporelle où l'évaporation et les besoins cumulatifs des plantes imposent une vigilance constante pour éviter le dessèchement des cultures. L'expérience de jeu est enrichie par un environnement vivant et parfois hostile. Le joueur doit faire face à des perturbations extérieures sous la forme d'animaux nuisibles (poules, corbeaux) régis par des algorithmes de recherche de chemin (pathfinding). Ces menaces oblige le joueur à surveiller l'intégrité de ses parcelles et à réagir rapidement pour faire partir les prédateurs. Enfin, l'objectif à long terme est d'assurer la croissance économique de l'exploitation. À travers un système de quêtes (principales et répétitives) et de gain d'expérience, le joueur progresse par niveaux, débloquent ainsi de nouvelles variétés de plantes, des infrastructures plus performantes et la possibilité de recruter une main-d'œuvre supplémentaire. D'un point de vue technique, ce projet vise à mettre en application le motif architectural MVC ainsi que la gestion des threads pour séparer les animations du reste de la logique de jeu et faire fonctionner plusieurs jardiniers ou animaux en parallèle.

2 Analyse

2.1 Analyse Globale

Le projet a été découpé en 8 fonctionnalités principales. Chaque fonctionnalité a été évaluée selon sa complexité technique et son importance critique pour la jouabilité minimale.

Fonctionnalité	Description	Difficulté	Priorité
F1. Terrain & Grille	Gestion du maillage, génération du terrain, système de coordonnées, cases, et vue avec scrolling	Moyenne	Élevée
F2. Gestion des Jardiniers	Déplacement (pathfinding), files d'actions, interruptions et gestion multi-agents (Threads).	Difficile	Élevée
F3. Cycle Végétal & Eau	Plantation, simulation de la croissance, absorption de l'eau, récolte, pourriture, assèchement	Moyenne	Élevée
F4. Constructions	Parcelles, Grange, Arroseur, Épouvantail, barrières, et action de construction	Difficile	Moyenne
F5. Ennemis	Pathfinding et comportements spécifiques (Poules, Corbeaux).	Moyenne	Basse
F6. Économie & Progression	Système de monnaie, inventaire, quêtes, expérience et déblocage de contenu.	Moyenne	Moyenne
F7. Interface & Contrôles	Gestion des clics, hotbar, menus contextuels, sélection dynamique d'objets, retours visuels	Difficile	Élevée
F8. Menu & Sauvegarde	Menu, création, chargement et sauvegarde de parties	Difficile	Moyenne

Table 1 – Tableau récapitulatif des fonctionnalités

2.2 Analyse Détaillée

F1 : Terrain et Grille

Le monde du jeu repose sur une structure matricielle (Grille) qui sert de référentiel unique pour tous les objets.

- **Création et initialisation de la grille** : Création du terrain et des objets initiaux.
- **Gestion des différents types de case** : Cases vides, plantables, obstacles
- **Vue principale** : Affichage graphique du terrain.
- **Scrolling de la vue** : Décaler la caméra lorsque la souris se rapproche des bords de l'écran

F2 : Gestion des Jardiniers

Cette fonctionnalité constitue le corps principal de la programmation concurrente du projet.

- **Actions** : Un jardinier se verra assigner des actions (labourer, planter, arroser ou récolter, chacune avec un comportement spécifique).
- **File d'actions** : Pour qu'un jardinier suive les actions qui lui sont assignées, il a une file d'actions ordonnée chronologiquement pour indiquer ses prochaines actions, changement du monde, fermeture du jeu...) et doit gérer cela sans problème
- **Déplacement et Pathfinding** : Le déplacement d'un point A à un point B nécessite un algorithme de recherche de chemin pour contourner les obstacles.
- **Inventaire** : Chaque jardinier aura un inventaire virtuel dans lequel il stockera les ressources dont il a besoin pour les actions à effectuer.
- **Comportement idle** : un jardinier qui n'a pas d'action à faire se balade périodiquement
- **Attribution des actions** : Les jardiniers se répartissent automatiquement les différentes tâches données par le joueur

F3 : Cycle Végétal et Eau

La gestion des plantes repose sur une mécanique temporelle et des états finis.

- **Cycle végétal** : Chaque plante a plusieurs états possibles : graine, pousse, en croissance, prête à récolter, morte, mangée et change d'état en fonction de l'environnement
- **Plantation/Récolte** : Un jardinier peut planter des graines sur une parcelle si elle est labourée. Un jardinier peut récolter une plante si elle est prête à être récolter ou morte (nettoyage de la case)
- **Arrosage** : Un jardinier peut arroser une plante en croissance, il faut cependant bien contrôler l'arrosage pour ne pas dépasser le seuil d'eau supporté par la plante
- **Gestion d'eau** : La plante consomme de l'eau pendant sa croissance, demandant un arrosage régulier. Si la plante n'a pas d'eau pendant trop longtemps, elle meurt.

F4 : Constructions

Plusieurs constructions seront utilisées dans le jeu, chacune avec son utilité propre. Certaines seront présentes depuis le début et permettront aux jardiniers d'effectuer leurs actions, tandis que d'autres pourront être construites par les jardiniers et joueront un rôle actif

dans la ferme.

- **Action construction** : Le joueur peut construire des bâtiments directement depuis le menu de construction, en payant leur prix. Certains bâtiment ont une limite au nombre de fois ou ils peuvent être placés
- **Action suppression** : Le joueur peut supprimer des bâtiments et obstacles naturels, regagnant une partie de leur coût en or.
- **Grange** : Point d'accès à l'inventaire de la ferme. C'est ici que les jardiniers iront pour déposer ou récupérer des graines, plantes ou autres objets. Le joueur peut y acheter/vendre des graines et vendre ses récoltes. Elle est construite par le joueur au début du tutoriel.
- **Parcelle** : Regroupe plusieurs cases plantables adjacentes, recalculée dynamiquement. Le joueur peut sélectionner une parcelle entière d'un coup pour donner des actions aux jardiniers. Peut être construite par un jardinier en labourant.
- **Barrière (optionnel)** : Bloque le passage dans les cases qu'elle recouvre, notamment celui des poulets. Peut être construite par le joueur. Débloquée plus tard dans le jeu. A une variante "porte" par laquelle les jardiniers peuvent passer mais pas les ennemis.
- **Irrigateur automatique (optionnel)** : Arrose automatiquement les plantes autour. Peut être construit par le joueur. Débloqué plus tard dans le jeu.
- **Décorations** : Divers bâtiments sans utilité pour le jeu à part la décoration. Débloquées progressivement

F5 : Ennemis

Pour dynamiser la partie, des entités hostiles apparaissent périodiquement. Tous partent du même moteur de pathfinding que les jardiniers mais avec des objectifs distincts :

- **Poulets** : Ciblent la plante la plus proche pour la manger. Partent après avoir mangé trois plantes. Fuient si le joueur leur clique dessus. Bloqués par les barrières.
- **Corbeaux** : Comme les poulets mais attaquent par les airs, non affectés par les barrières. Fuient si le joueur leur clique dessus ou si rentrent dans le rayon d'action d'un épouvantail. Apparaissent seulement après un certain niveau.

F6 : Économie et Progression

Cette couche logique gère la boucle de progression et de récompenses. Et pour rendre le jeu plus attrayant, il inclut un système d'économie.

- **Expérience** : Le joueur gagne de l'expérience en récoltant des plantes et en finissant

des quêtes.

- **Quêtes** : Les quêtes sont visibles depuis le menu de quêtes, divisé en 6 chapitres. Les chapitres 1 à 5 sont les quêtes principales, poussant le joueur à explorer toutes les fonctionnalités du jeu. Le chapitre 6 contient des quêtes répétitives, générées automatiquement (optionnel)
- **Débloqué** : Le gain d'expérience permet de monter en niveau. A chaque niveau débloqué, le système notifie l'interface afin d'afficher les nouvelles ressources disponibles (nouvelles graines, bâtiments et ennemis).
- **Economie** : Le joueur va devoir gérer son argent qui lui permettra d'acheter des graines et bâtiments... Les plantes récoltées peuvent être vendues afin de progresser.

F7 : Interface et Contrôles

Le jeu aura 3 modes principaux : le mode Global, où la caméra peut se déplacer et le joueur peut lancer les différents menus d'action, le mode Popup où le joueur pourra consulter des informations et faire des actions, et le mode Sélection, où le joueur devra sélectionner une ou des cases spécifiques du monde demandées par un popup.

- **Popups informatifs** : Le joueur aura l'option de voir des informations sur différents objets (bâtiments, graines). Ces popups peuvent être ouverts et fermés, et contiennent uniquement du texte.
- **Feedback visuel** : De petits indicateurs visuels apparaîtront à l'écran lors de la récolte, de la vente ou de l'achèvement de quêtes, affichant la quantité d'expérience et de PO gagnés.
- **Sélecteur d'actions** : Affiche en bas de l'écran pendant le mode global, contient les 4 actions que le joueur peut donner aux jardiniers.
- **Sélecteur Inventaire** : Popup affichant l'inventaire de la ferme et permettant de choisir un type d'objet (graines par ex lors de l'action planter)
- **Popup Grange** : Affiche l'inventaire de la grange (l'inventaire central de la ferme). Permet de consulter les stocks, d'avoir des informations sur les différents items, et acheter/vendre des items
- **Mode sélection** : Mode affichant le monde avec un texte explicatif par dessus, et permettant au joueur de sélectionner une ou plusieurs cases avec la souris du type demande (ex : zone plantable). Clic = sélectionner / désélectionner une case, shift+clic = sélectionner une parcelle entière, entrée = confirmer, esc = annuler.
- **Feedback sélection** : Les cases sélectionnées seront mises en évidence par un surlignage bleu. Une fois la sélection validée, les cases sur lesquelles une action est

prévue seront mises en évidence par un surlignage jaune. Si plusieurs jardiniers ont planifié une action sur la même case, l'intensité du jaune augmentera pour refléter le nombre d'interventions prévues.

- **Panneau de quêtes** : S'affiche à gauche de l'écran quand le joueur clique sur l'icône "quetes" pendant le mode global. Contient les descriptifs de toutes les quêtes, organisées par chapitres.
- **Panneau de construction** : S'affiche à droite de l'écran quand le joueur clique sur l'icône "construction" pendant le mode global. Contient tous les types de bâtiments classés par catégories, avec icône, description et bouton "poser" si débloqués, icône de cadenas sinon. Poser un bâtiment enchaîne avec le mode construction.
- **Mode construction** : Variante du mode sélection où le joueur doit placer un bâtiment sur un espace disponible. Une version fantôme du bâtiment apparaît et suit la souris, facilitant le placement.
- **Mode suppression** : Variante du mode sélection où le joueur doit sélectionner un ou plusieurs bâtiments ou obstacles, puis est face à un popup de confirmation pour supprimer les bâtiments. Déclenché en cliquant l'icône "poubelle" dans le mode global.
- **Tutoriel (optionnel)** : Suite de popups informatifs au début du jeu expliquant les notions principales et invitant le joueur à les prendre en main

F8 : Menu et Sauvegarde

Au démarrage du jeu, le joueur interagit avec un menu afin de soit lancer une nouvelle partie soit lancer une sauvegarde.

- **Écran d'accueil** : Interface initiale permettant de naviguer entre la création d'une nouvelle exploitation, la reprise d'une partie existante ou la configuration des options.
- **Création d'une partie** : Initialisation d'une partie avec les paramètres par défaut (ressources de départ, premier jardinier, terrain vierge).
- **Système de sauvegarde** : Chaque sauvegarde stocke l'état du terrain, de l'économie et de la progression.
- **Gestion des sauvegardes** : Il est possible de supprimer et de renommer les sauvegardes existantes.
- **Paramètre (optionnel)** : Gestion de la musique, des effets sonores, ...

3 Plan de Développement

3.1 Répartition et charge de travail

Le projet mobilise une équipe de 4 développeurs pour un volume horaire total estimé à 150 heures. Ce volume de travail permet de répartir les tâches entre les membres de l'équipe, mais aussi de définir une architecture claire, de travailler sur le rendu et les déplacements et de limiter les problèmes liés aux threads.

F1. Terrain & Grille (25h)

- **Analyse (4h) :**
 - **Structure de données :** Choix d'une structure adaptée pour stocker la carte.
 - **Système de coordonnées :** Conversions mathématiques entre la position à l'écran (pixels) et les cases logiques du jeu.
 - **Scrolling :** Réflexion sur l'implémentation du scrolling.
- **Conception (5h) :**
 - **Architecture logicielle :** Découpage clair séparant la logique du jeu, l'affichage et les contrôles utilisateur.
 - **Typologie du terrain :** Définition des différents types de sols (vide, terre, obstacle) et de leurs propriétés.
 - **Système de caméra :** Conception de la logique de défilement (scrolling) et gestion du décalage de la vue.
- **Développement (12h) :**
 - **Génération de la carte :** Programmation de la création du terrain initial et de son remplissage.
 - **Moteur d'affichage :** Développement de la boucle de rendu pour dessiner uniquement les éléments visibles à l'écran.
 - **Déplacement de vue :** Implémentation de la détection de la souris près des bords pour bouger la caméra.
 - **Interactions :** Programmation de la sélection, permettant d'identifier quelle case est ciblée par le joueur.
- **Tests (4h) :**
 - **Tests de limites :** Vérification de la stabilité du jeu lorsque la caméra ou la souris atteint les bords de la carte.

- **Tests de fluidité** : Validation de la rapidité d’affichage lors des déplacements rapides.
- **Validation fonctionnelle** : Confirmation que la zone cliquée à l’écran correspond à la case enregistrée en mémoire.

F2. Gestion des Jardiniers (30h)

- **Analyse (4h)** :
 - **Algorithmes de déplacement** : Méthodes de recherche de chemin A* pour contourner les obstacles efficacement.
 - **Stratégie de concurrence** : Étude de la gestion simultanée de plusieurs Threads sans ralentir l’interface utilisateur.
 - **Gestion des ressources partagées** : Identification des données critiques (ex : une case de terrain) nécessitant une protection contre les accès simultanés.
- **Conception (6h)** :
 - **Comportement des entités** : modélisation des différents états possibles d’un jardinier (En attente, En déplacement, Au travail) sous forme de diagramme.
 - **Système de tâches** : Conception de la file d’attente permettant de stocker et d’ordonner les actions demandées par le joueur.
 - **Architecture de synchronisation** : Définition des mécanismes de verrouillage pour éviter les conflits lorsque deux jardiniers visent la même parcelle.
- **Développement (15h)** :
 - **Moteur de déplacement** : Programmation de la logique de calcul d’itinéraire point par point en évitant les murs et obstacles.
 - **Cycle de vie** : Implémentation de la boucle principale de l’entité qui vérifie sa liste de tâches et agit en conséquence.
 - **Exécution des actions** : Codage des interactions concrètes (labourer, planter, récolter) et de leur durée associée.
 - **Gestion de l’inventaire** : Développement de la logique de transport de ressources par les jardiniers.
- **Tests (5h)** :
 - **Tests de charge** : Simulation avec un grand nombre de jardiniers pour vérifier la stabilité du système.

- **Tests de collision** : Vérification que les jardiniers ne se bloquent pas entre eux ou dans les obstacles.
- **Validation des priorités** : Contrôle que l'annulation d'une tâche ou l'interruption fonctionne instantanément.

F3. Cycle Végétal & Eau (15h)

- **Analyse (2h)** :
 - **Règles biologiques** : Définition des paramètres de croissance pour chaque variété (durée, résistance à la sécheresse).
 - **Simulation hydraulique** : Étude du modèle d'évaporation et de l'impact du manque d'eau sur la santé des cultures.
 - **Cycles de vie** : Identification des différentes phases d'évolution, de la semence à la décomposition.
- **Conception (3h)** :
 - **Machine à états** : modélisation des transitions entre les stades (Graine -> Pousse -> Mature -> Fané).
 - **Gestion du temps** : Conception du mécanisme de mise à jour périodique (ticks) qui orchestre le vieillissement des plantes et l'assèchement des sols.
 - **Interaction Sol-Plante** : Définition de la logique de consommation d'eau par la plante.
- **Développement (8h)** :
 - **Logique de croissance** : Programmation des minuteurs et des conditions de passage d'un stade à l'autre.
 - **Système d'irrigation** : Implémentation de la baisse progressive du niveau d'eau et de sa recharge (action d'arroser).
 - **Gestion des états critiques** : Codage des conséquences de la négligence (ex : arrêt de croissance, mort de la plante).
 - **Mise à jour visuelle** : Changement dynamique de l'apparence de la plante selon sa maturité.
- **Tests (2h)** :
 - **Simulation accélérée** : Validation du cycle complet d'une plante en réduisant artificiellement les durées.

- **Tests de contrainte** : Vérification du comportement lorsqu'une plante n'est jamais arrosée (doit mourir/faner).
- **Cohérence visuelle** : Contrôle de la correspondance entre l'état logique de la plante et son affichage à l'écran.

F4. Constructions (16h)

- **Analyse (2h)** :
 - **Typologie des structures** : Définition des bâtiments disponibles (puits, grange, épouvantails) et de leurs fonctions respectives.
 - **Modèle économique** : Établissement des coûts de construction et des prérequis (niveau du joueur, ressources nécessaires).
 - **Contraintes spatiales** : Étude des dimensions de chaque structure et des règles de placement (terrain vide requis).
- **Conception (2h)** :
 - **Mécanique de placement** : Conception du mode "Construction" permettant de prévisualiser l'emplacement avant validation.
 - **Modification dynamique** : Définition de la logique qui transforme une case "traversable" en "obstacle" infranchissable pour les entités.
 - **Hierarchie des objets** : Organisation des classes pour différencier les bâtiments interactifs (Puits) des obstacles passifs (Barrières).
- **Développement (10h)** :
 - **Intégration du mode construction** : Programmation de l'interface de sélection et de la validation du positionnement par la souris.
 - **Gestion des obstacles** : Mise à jour en temps réel de la grille de navigation pour que les algorithmes de déplacement prennent en compte les nouveaux murs.
 - **Interactions spécifiques** : Implémentation des fonctions liées aux bâtiments (ex : recharger l'eau au puits).
- **Tests (2h)** :
 - **Validation des collisions** : Vérification stricte que les jardiniers et ennemis contournent bien les nouvelles constructions.
 - **Vérification économique** : Contrôle du débit des ressources lors de l'achat et

impossibilité de construire sans fonds.

- **Tests de superposition** : Garantie qu'il est impossible de construire deux bâtiments au même endroit.

F5. Ennemis (15h)

- **Analyse (2h)** :
 - **Typologie des menaces** : Classification des nuisibles (terrestres vs aériens) et de leurs impacts spécifiques sur le terrain (destruction, vol, altération).
 - **Algorithmes de ciblage** : Étude des règles de priorité pour l'Intelligence Artificielle (cibler la plante la plus proche ou une zone aléatoire).
 - **Courbe de difficulté** : Réflexion sur la fréquence d'apparition des ennemis pour maintenir un défi progressif sans frustrer le joueur.
- **Conception (3h)** :
 - **Modélisation de l'IA** : Conception des arbres de comportement pour chaque type d'ennemi (Recherche -> Attaque -> Fuite).
 - **Adaptation du déplacement** : Distinction entre les unités soumises aux obstacles (murs) et celles capables de les ignorer (vol).
 - **Système de contre-mesures** : Définition des moyens de défense à disposition du joueur (clic répulsif, épouvantails).
- **Développement (8h)** :
 - **Moteur comportemental** : Implémentation des routines spécifiques (le lapin mange, la taupe creuse, le corbeau vole).
 - **Gestion des dégâts** : Programmation des conséquences d'une attaque réussie sur les parcelles (disparition de la plante, case inutilisable).
 - **Interactions défensives** : Codage de la détection des clics sur les ennemis et de leur réaction (fuite ou disparition).
 - **Génération de vagues** : Mise en place du système d'apparition aléatoire en bordure de carte.
- **Tests (2h)** :
 - **Vérification des trajectoires** : Contrôle que les unités volantes passent bien au-dessus des obstacles tandis que les terrestres les contournent.

- **Tests de neutralisation** : Validation de l'efficacité des actions du joueur pour repousser les menaces.
- **Validation des priorités** : Garantie que les ennemis ciblent bien les éléments prévus et non des zones vides.

F6. Économie & Progression (10h)

- **Analyse (2h)** :
 - **Modélisation économique** : Équilibrage des coûts d'achat (graines, structures) et des revenus de vente pour assurer une rentabilité progressive.
 - **Courbe de progression** : Calcul des paliers d'expérience (XP) nécessaires pour monter de niveau et définition du rythme de déblocage.
 - **Système d'objectifs** : Distinction entre les quêtes narratives et les tâches répétitives.
- **Conception (2h)** :
 - **Gestionnaire de ressources** : Conception de l'entité centrale responsable du suivi des avoirs globaux (argent, niveau actuel).
 - **Arbre de déblocage** : Définition des prérequis pour chaque nouvel élément (ex : telle plante nécessite le niveau 3).
 - **Flux de récompense** : Planification des notifications visuelles lors de l'accomplissement d'une quête ou d'un passage de niveau.
- **Développement (6h)** :
 - **Suivi des transactions** : Programmation des mécanismes d'achat et de vente avec vérification des fonds disponibles.
 - **Gestion de l'expérience** : Implémentation de l'incrémentation de l'XP et du déclenchement automatique des montées de niveau.
 - **Restriction d'accès** : Codage de la logique qui empêche ou autorise l'utilisation d'objets selon le niveau du joueur.
- **Tests (2h)** :
 - **Tests de limites** : Vérification stricte de l'impossibilité d'effectuer un achat sans les fonds nécessaires (pas de solde négatif).
 - **Validation des déblocages** : Contrôle que les nouveaux items deviennent bien accessibles immédiatement après le passage de niveau.

- **Cohérence des gains** : Vérification que les montants crédités correspondent bien aux valeurs définies.

F7. Interface & Contrôles (25h)

- **Analyse (3h)** :
 - **Expérience utilisateur** : Création de maquettes pour définir l'agencement des menus, des barres d'état et des fenêtres contextuelles.
 - **Flux de contrôle** : Définition des interactions possibles (clic gauche pour sélectionner, clic droit pour action, raccourcis clavier).
 - **Hiérarchie visuelle** : Organisation des informations pour prioriser l'affichage des données vitales (eau, argent) sans surcharger l'écran.
- **Conception (5h)** :
 - **Architecture événementielle** : Conception du système qui capte les entrées (souris/clavier) et les transmet au contrôleur du jeu.
 - **Système de notification** : Modélisation du mécanisme permettant à l'interface de se mettre à jour automatiquement dès qu'une donnée change dans le modèle.
 - **Menus contextuels** : Design logique des fenêtres flottantes qui apparaissent lors de la sélection d'une entité.
- **Développement (12h)** :
 - **Gestion des entrées** : Programmation de la détection précise des clics sur les entités mouvantes ou statiques.
 - **Panneaux d'information** : Implémentation de l'affichage permanent des ressources globales et du temps.
 - **Interactivité** : Développement des boutons d'action et des retours visuels (surbrillance au survol).
 - **Fenêtres modales** : Création des pop-ups pour les magasins, l'inventaire ou les messages d'alerte.
- **Tests (5h)** :
 - **Tests d'ergonomie** : Vérification de la facilité de prise en main et de la clarté des menus.
 - **Réactivité** : Contrôle de l'absence de latence entre le clic et l'action (feedback immédiat).

- **Adaptabilité** : Validation du bon affichage de l'interface lors du redimensionnement de la fenêtre de jeu.

F8. Menu & Sauvegarde (10h)

- **Analyse (2h)** :
 - **Stratégie de persistance** : Étude des formats de fichiers pour stocker l'état complet du jeu (terrain, économie, progression) de manière sécurisée.
 - **Flux utilisateur** : Définition de l'arborescence des écrans (Accueil, Jeu, Pause, Options) et des transitions entre eux.
 - **Gestion de l'intégrité** : Analyse des risques liés à la sauvegarde.
- **Conception (2h)** :
 - **Architecture de données** : Modélisation de l'objet Sauvegarde qui regroupe toutes les variables nécessaires pour reconstruire une partie à l'identique.
 - **Design des interfaces** : Conception graphique des écrans de menu principal et de pause pour qu'ils soient cohérents avec le reste du jeu.
 - **Logique de suspension** : Définition du mécanisme de Pause qui doit figer tous les threads actifs avant l'écriture sur le disque.
- **Développement (6h)** :
 - **Implémentation des menus** : Programmation de la navigation et des boutons (Nouvelle Partie, Charger, Quitter).
 - **Système de sérialisation** : Développement des fonctions d'écriture (Sauvegarder) et de lecture (Charger) des fichiers de données.
 - **Gestion des paramètres** : Codage des options configurables (volume sonore, résolution) et de leur mémorisation entre deux lancements.
- **Tests (2h)** :
 - **Tests d'intégrité** : Vérification que le chargement d'une partie restitue exactement l'état au moment de la sauvegarde (position des entités, état des plantes).
 - **Validation de la pause** : Contrôle que le jeu reprend bien sans décalage temporel après une interruption.
 - **Robustesse** : Essais de chargement de fichiers corrompus ou inexistants pour s'assurer que l'application ne plante pas.

4 Conception Générale

4.1 Architecture Globale

L'architecture de notre application repose sur le patron de conception MVC (Modèle-Vue-Contrôleur). Ce paradigme a été choisi pour assurer une séparation entre la logique métier, les données de la simulation et l'interface graphique. Ce découpage rend le code plus facile à modifier et à tester.

- **Le Modèle (Model)** : Centralisé autour de la classe `World`, il incarne le cœur logique de la simulation. Il encapsule l'intégralité de l'état du jeu à un instant donné, incluant la matrice spatiale des parcelles (`Tile`), l'inventaire des ressources du joueur (fonds financiers, réserves d'eau, stocks de graines), ainsi que le registre de toutes les entités dynamiques en cours d'exécution.
- **La Vue (View)** : Principalement orchestrée par les classes `Display` et `Rendering`, cette composante gère le rendu visuel en s'appuyant sur la bibliothèque Java Swing. Elle a pour rôle de transcrire continuellement les données brutes du Modèle en une interface riche et interactive, gérant la superposition des calques de terrain, l'animation des sprites et l'affichage des surcouches d'information (menus et indicateurs).
- **Le Contrôleur (Controller)** : Piloté par la classe `GameManager` et soutenu par les écouteurs d'événements (interactions clavier et clics de souris), il agit comme l'interface de commande. Il capte les intentions du joueur (comme la sélection d'une parcelle à labourer ou le déplacement de la caméra), les traduit en requêtes applicatives, et orchestre en conséquence la mise à jour sécurisée des états au sein du Modèle.

4.2 Schéma de l'Architecture Fonctionnelle

Voici le schéma détaillé des flux d'informations et des responsabilités :

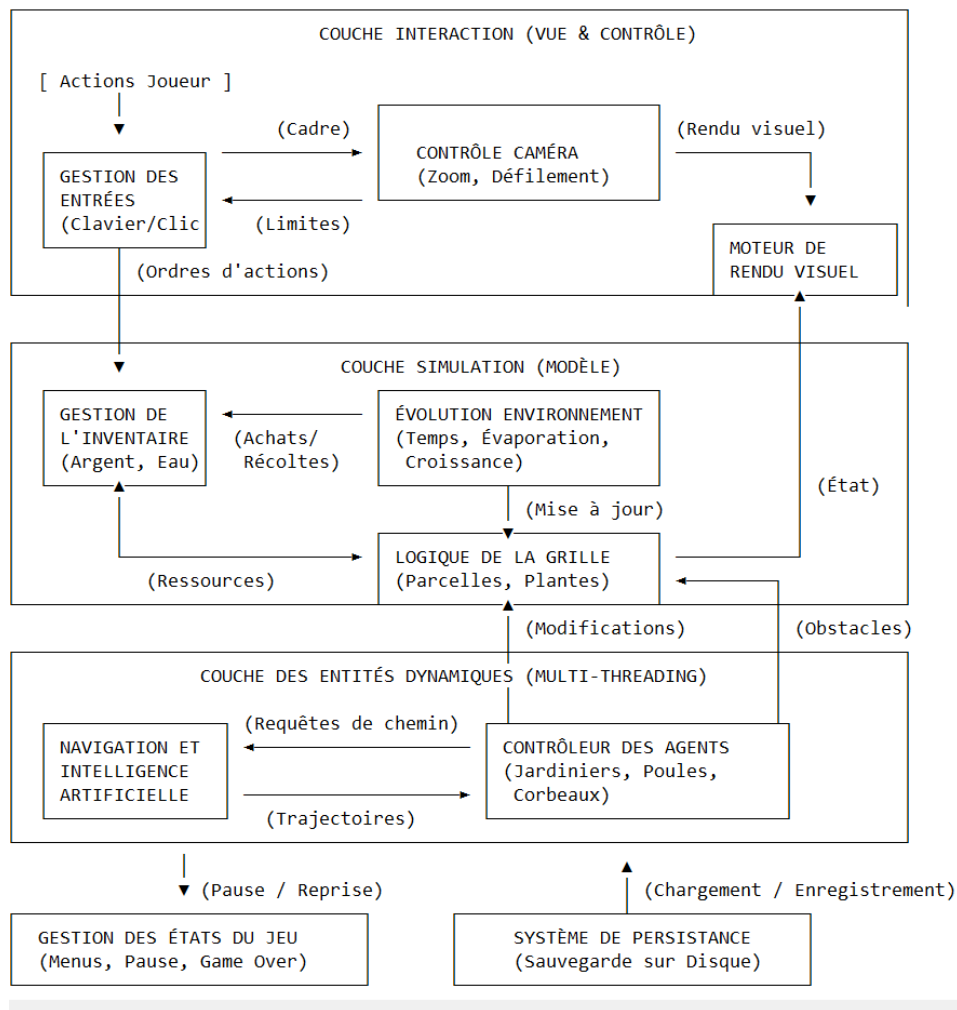


Figure 2 – Schéma de l'Architecture Fonctionnelle

4.3 Description Détaillée des Blocs Fonctionnels

Couche Interaction (Ce que le joueur perçoit et manipule)

- **Gestion des Entrées** : C'est le point d'entrée du joueur. Ce bloc capte les événements matériels (clics de souris sur une case, touches du clavier) et les traduit en intentions claires (ex : "planter une graine à cette coordonnée", "déplacer la vue vers le haut").
- **Contrôle de la Caméra** : Il maintient une "fenêtre de vue" virtuelle. Il calcule quelle portion du terrain global doit être dessinée en fonction du niveau de zoom et des déplacements demandés par le joueur, et gère les limites pour ne pas sortir de la carte.
- **Moteur de Rendu Visuel** : Animé par une boucle temporelle rapide (60 images par seconde), ce bloc superpose les différentes couches graphiques (le terrain de fond, les objets, les animaux, puis les menus par-dessus). Il interroge en permanence le reste du système pour dessiner l'état exact du jeu à l'instant t.

Couche Simulation

- **Logique de la Grille** : Ce bloc gère la mémoire spatiale du jeu. Il connaît l'état individuel de chaque parcelle de terre (labourée, plantée, sèche, humide) et traite les demandes de modification qui lui parviennent.
- **Évolution de l'Environnement** : Il gère le passage du temps de manière globale. Il applique les règles physiques du jeu de manière régulière : diminution du niveau d'eau dans la terre (évaporation) et avancement des stades de croissance des plantations.
- **Gestion de l'Inventaire et Économie** : Ce petit bloc sécurisé centralise les ressources du joueur. Il s'assure qu'une action n'est validée que si les fonds (argent, eau dans le réservoir, graines) sont suffisants.

Couche des Entités Dynamiques

- **Contrôleur des entités** : Ce bloc gère le cycle de vie de chaque entité mobile (jardiniers, animaux). Chaque entité fonctionne de manière autonome en parallèle du reste du jeu. Ils observent l'état de la grille et décident de leurs actions (arroser, fuir, manger une récolte).
- **Navigation & Pathfinding** : Appelé par le contrôleur des entités, ce bloc purement mathématique analyse la grille pour calculer les trajets optimaux. Il permet aux entités de contourner les obstacles ou de trouver la cible la plus proche.

Couches Transversales

- **Gestion des États du Jeu** : Cette fonctionnalité vise une navigation fluide et intuitive entre les sections et options du jeu. Elle permet aux joueurs de se déplacer aisément entre les menus principaux (Démarrer la Partie, Menu Pause). Une bonne navigation est essentielle pour l'expérience utilisateur, réduisant la frustration et assurant un accès rapide aux informations.
- **Système de Persistance** : Invoqué manuellement ou automatiquement, il fige l'ensemble des données de la Couche Simulation et de la Couche des Entités pour les écrire dans un fichier, ou fait l'opération inverse pour recharger une partie en cours.

5 Conception détaillée

5.1 Fonctionnalité 1 : Terrain et Grille

La gestion du terrain constitue le socle de notre simulation. L'espace de jeu y est modélisé sous la forme d'une grille spatiale discrète, une structure indispensable tant pour permettre l'interaction précise du joueur que pour faciliter la navigation algorithmique des entités

autonomes.

5.1.1 Création et initialisation de la grille : Terrain et objets initiaux

Structures de données et constantes Pour modéliser l'espace physique de la simulation, le système s'appuie sur une structure de données matricielle. L'environnement est représenté par un tableau bidimensionnel d'objets `Tile`, encapsulé au sein de la classe principale gérant le monde. Les dimensions de cette grille sont dictées par des constantes structurelles définissant la largeur et la hauteur maximales de la carte (ex : `MAP_WIDTH`, `MAP_HEIGHT`). De plus, l'état initial de chaque parcelle s'appuie sur les définitions constantes d'une énumération stricte (`TileType`), répertoriant les natures de terrain possibles telles que l'herbe, la terre nue ou les obstacles.

Algorithme d'initialisation Le processus de création du monde suit un algorithme séquentiel exécuté au démarrage de la partie. Dans un premier temps, le système alloue l'espace mémoire nécessaire pour la matrice en fonction des dimensions requises. Ensuite, un parcours itératif complet (via une double boucle sur les axes des abscisses et ordonnées) instancie un objet `Tile` pour chaque coordonnée de la grille. Durant ce parcours, une logique de zonage spatial est appliquée : l'algorithme attribue de l'herbe à toutes les cases, puis ajoute des obstacles aléatoirement sur certaines.

Conditions limites et robustesse Le système doit s'assurer en amont que les dimensions fournies pour l'allocation de la matrice sont strictement positives. Lors de la génération et des accès ultérieurs, toute tentative d'itération en dehors de l'intervalle strict des indices de la grille doit être interceptée pour prévenir les erreurs de dépassement (`OutOfBounds`). Enfin, il est impératif de vérifier, via des tests unitaires, qu'aucune coordonnée de la matrice ne demeure non instanciée à l'issue de l'algorithme, afin d'éviter tout plantage critique lors du premier cycle de simulation ou de rendu graphique.

Utilisation par les autres fonctionnalités Une fois instanciée, cette matrice centralisée agit comme la source de vérité unique pour le reste de l'application. Le système de sauvegarde parcourt cette structure pour capturer l'état complet du terrain et le sérialiser dans un fichier. Les modules gérant les entités autonomes l'interrogent en continu pour analyser la perméabilité des cases (notamment pour le calcul de trajectoire via `pathfinding`) ou pour localiser des cibles. Enfin, la couche visuelle lit cette matrice à chaque rafraîchissement d'image pour dessiner la carte à l'écran.

5.1.2 Gestion des différents types de cases

Structures de données et constantes La caractérisation physique et logique de chaque parcelle repose sur une conception orientée objet stricte, avec un type `Tile` de base comportant des méthodes définissant son comportement : `isWalkable()`, `isFarmable()`, `isPlowable()`. Il existe une sous-classe `PlantTile` qui représente une case labourée et contient en plus des informations sur la plante qui y est plantée.

Conditions limites et robustesse La gestion des états des cases nécessite des mécanismes de sécurité rigoureux pour maintenir la cohérence de la simulation. Il est impératif de tester et de bloquer les transitions invalides, telles qu'une tentative de récolte sur une parcelle d'herbe ou de labourage sur le réservoir d'eau. De plus, dans le contexte d'une application multithreadée, la robustesse du système repose sur la sécurisation des accès concurrents. Si le joueur et une entité autonome (comme un corbeau) tentent de modifier l'état de la même case simultanément (par exemple, récolter et manger la même plante à la milliseconde près), l'opération de modification de l'état est conçue pour éviter tout conflit de mémoire ou incohérence logique.

Utilisation par les autres fonctionnalités Cette gestion des états est un rouage essentiel qui irrigue l'ensemble des autres modules de l'application. L'interface utilisateur l'exploite en continu pour déterminer le contexte d'interaction, l'action "Récolter" est possible uniquement si la case survolée est au stade requis. Le module de navigation s'en sert pour classer les parcelles : les obstacles infranchissables sont exclus des calculs de trajectoire, tandis que les cultures matures sont identifiées comme des cibles. Enfin, le moteur de rendu interroge cet état à chaque image pour superposer la texture et les sprites adéquats.

5.1.3 Création de la vue principale : Affichage du terrain

Structures de données et constantes La matérialisation visuelle du monde repose sur une architecture de rendu découplée, principalement portée par les classes Display, Global et Rendering. Les structure de données centrales utilisées pour l'affichage sont le JPanel et le Graphics2D fourni par la bibliothèque Java Swing, qui permet de manipuler les primitives graphiques et les images. Le moteur de rendu s'appuie sur des constantes techniques fondamentales : `TILE_SIZE`, qui définit la dimension fixe en pixels d'une parcelle à l'écran (déterminant ainsi l'échelle du jeu), et FPS (Frames Per Second), fixée généralement à 60 pour assurer une fluidité visuelle suffisante tout en gardant de bonnes performances. Les ressources graphiques (textures de terre, d'herbe, sprites de plantes) sont pré-chargées dans des structures de type Image pour éviter tout accès disque coûteux durant le cycle d'affichage.

Algorithme de rendu du terrain L'affichage du terrain est géré par une boucle de rendu dans un thread cadencé par un Timer. À chaque itération, l'algorithme procède à un balayage systématique de la matrice du monde. Pour chaque cellule (x, y) de la grille, l'algorithme calcule sa position réelle à l'écran en appliquant une transformation linéaire : les coordonnées matricielles sont multipliées par la constante `TILE_SIZE`, puis corrigées par les variables de décalage générées par le contrôleur de caméra. Une fois la position cible déterminée, le système effectue un rendu par couches successives. Il dessine d'abord la texture de fond correspondant au `TileType` de la parcelle, puis superpose l'image de la culture si un stade de croissance est actif. Cette approche itérative garantit que l'utilisateur perçoit instantanément toute modification logique du modèle (comme le passage d'une terre sèche à une terre humide) sous forme de changement visuel.

Conditions limites et robustesse La robustesse de l’affichage dépend de sa capacité à gérer des scènes potentiellement vastes sans dégradation des performances. L’algorithme intègre une condition d’élagage : avant de lancer l’opération de dessin d’une case, il vérifie si ses coordonnées finales à l’écran sont comprises dans le rectangle de vue de la fenêtre. Si une case est hors champ, l’opération est ignorée, économisant ainsi des ressources processeur. Par ailleurs, pour garantir la stabilité de l’application dans un environnement multithreadé, l’accès aux données du monde durant le rendu doit être protégé. Le système s’assure que le moteur de rendu travaille sur une vue cohérente des données, évitant qu’une modification de la structure de la grille en plein milieu d’un cycle de dessin ne provoque des artefacts visuels ou des erreurs de type `ConcurrentModificationException`.

Utilisation par les autres fonctionnalités Cette vue principale sert de point de contact visuel pour l’intégralité des modules interactifs. Elle traduit les états complexes du moteur de simulation en informations compréhensibles pour le joueur, lui permettant de diagnostiquer d’un coup d’œil les besoins de ses cultures. De plus, elle est intimement liée au contrôleur d’entrées : c’est la vue qui permet de convertir un clic de souris sur un pixel précis de l’écran en une coordonnée logique (x, y) dans la matrice du monde, permettant ainsi au joueur d’interagir avec précision sur l’environnement. Enfin, elle agit comme le réceptacle final pour l’affichage des jardiniers et animaux qui viennent se superposer au terrain déjà rendu.

5.1.4 Scrolling de la vue

Structures de données et constantes Le mécanisme de navigation visuelle, permettant d’explorer une carte plus vaste que l’écran, s’appuie sur un système de coordonnées relatives généralement piloté par un contrôleur de caméra (`CameraController`). Les données fondamentales de cette fonctionnalité se résument à des variables scalaires de translation spatiale, typiquement nommées `offsetX` et `offsetY`, qui mémorisent le décalage de la vue par rapport à l’origine absolue de la grille (la case 0,0). Les constantes et paramètres associés incluent la vitesse de déplacement de la caméra (ex : `speed`), qui définit le nombre de pixels parcourus par cycle d’actualisation, ainsi que les dimensions dynamiques de la fenêtre d’affichage nécessaires pour évaluer le champ de vision actuel.

Algorithme de déplacement Le défilement de la vue repose sur un algorithme hybride, combinant la gestion événementielle et une mise à jour continue. Dans un premier temps, des `KeyListeners` interceptent les pressions sur les touches directionnelles du clavier ou le mouvement de la souris sur les bords de l’écran. Ces événements sont traduits en vecteurs de vitesse (gauche, droite, haut, bas). Ensuite, lors de chaque itération de la boucle de contrôle de la caméra, l’algorithme applique cette vitesse aux variables de translation `offsetX` et `offsetY`. Cette modification progressive et continue des coordonnées de décalage permet d’obtenir un glissement fluide et régulier du terrain.

Conditions limites et robustesse Pour garantir une expérience utilisateur cohérente, l’algorithme intègre un mécanisme de bornage strict. Il est impératif d’empêcher la caméra

de défiler indéfiniment au-delà des limites physiques du monde généré, ce qui exposerait le joueur à un espace vide en dehors de la carte. À chaque mise à jour des coordonnées, l'algorithme vérifie que les valeurs de décalage ne dépassent pas un intervalle de sécurité.

Utilisation par les autres fonctionnalités Ces variables de translation sont critiques pour faire le lien entre le joueur et le modèle. Elles sont prioritairement consommées par le moteur de rendu qui, comme abordé dans la fonctionnalité précédente, ajoute ces décalages à chaque élément dessiné pour simuler le mouvement de la caméra. De plus, elles sont indispensables au module de gestion des clics souris. Lorsqu'une action est détectée, le système doit appliquer une transformation spatiale inverse : il soustrait les valeurs `offsetX` et `offsetY` aux coordonnées brutes en pixels du clic à l'écran, puis divise le résultat par la taille unitaire d'une case. Cette projection mathématique permet de remonter avec une précision absolue de l'espace de l'écran vers l'espace logique de la matrice, identifiant l'objet Tile exact ciblé par le joueur.

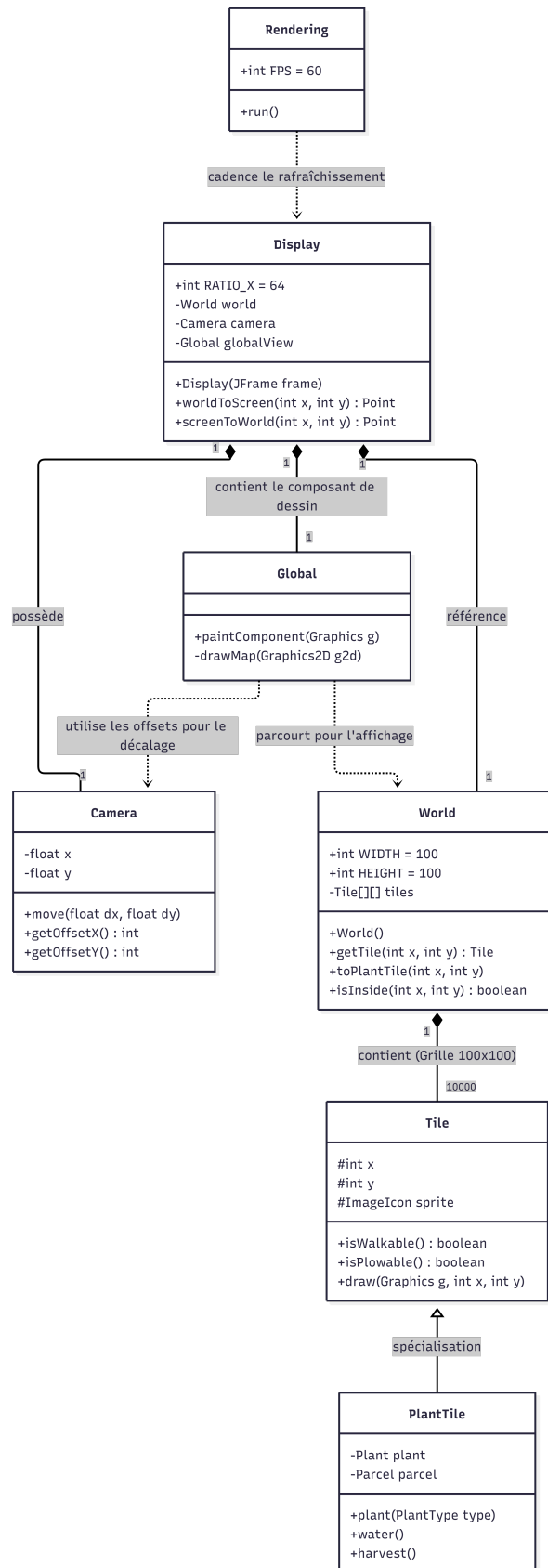


Figure 3 – Diagramme de classe de la fonctionnalité 5.1

5.2 Gestion des jardiniers

Les jardiniers sont les entités principaux de la simulation. Contrairement à une approche séquentielle classique, chaque jardinier est conçu comme une entité autonome capable d'agir indépendamment des autres et du cycle de rendu graphique. Nous avons décidé d'implémenter deux jardiniers, qui agissent parallèlement.

5.2.1 Le Cycle de Vie et la Programmation Concurrente

Structures de données et constantes Chaque jardinier est modélisé par une classe héritant de la super-classe abstraite `Entity` (mutualisant ainsi les données spatiales et l'algorithme de navigation A* partagé avec les ennemis) et implémente l'interface `Runnable`, ce qui lui permet d'être exécuté dans son propre Thread. Les structures de données internes incluent des variables d'état (état actuel : au repos, en déplacement, en travail), une référence vers la parcelle cible, et un identifiant unique. Les constantes associées à cette fonctionnalité définissent le rythme de vie de l'entité, notamment le `ACTION_DELAY` (temps de pause entre deux cycles de réflexion) et la vitesse de travail, exprimée en millisecondes, qui détermine la durée nécessaire pour accomplir une tâche de labourage ou de plantation.

Algorithme du cycle de vie : L'algorithme repose sur une boucle de contrôle infinie dans la méthode `run()`. Le thread utilise des mécanismes de synchronisation natifs de Java pour éviter la saturation du processeur.

- **Attente passive :** Si la file d'attente d'actions (`actionQueue`) est vide, le thread entre dans un bloc synchronisé et invoque la méthode `wait()`. Le jardinier s'endort totalement et ne consomme plus aucune ressource CPU.
- **Réveil et Évaluation :** Lorsqu'une commande est assignée par le joueur, la méthode d'ajout appelle `notify()`, ce qui réveille instantanément l'entité. Il extrait alors la première tâche de sa file.
- **Exécution et Temporisation :** Lors des phases de déplacement via l'algorithme A*, le thread utilise `Thread.sleep()` à chaque pas. Ce mécanisme ne sert pas de pause de réflexion, mais permet de synchroniser la vitesse d'exécution logique avec le défilement de l'animation visuelle à l'écran, rendant les mouvements fluides pour le joueur.

Conditions limites et robustesse : La gestion autonome impose une grande réactivité face aux changements d'avis du joueur. Si le joueur assigne une nouvelle tâche alors que le jardinier est en plein déplacement, le système lui envoie un signal d'interruption (`thread.interrupt()`). L'algorithme est conçu pour capturer proprement l'exception `InterruptedException` : il purge sa tâche en cours, réinitialise son état, et embraye immédiatement sur le nouvel ordre sans faire planter l'application.

Utilisation par les autres fonctionnalités Ce cycle de vie autonome communique directement avec le moteur de simulation pour mettre à jour la grille. Il est piloté par le contrôleur d'entrées (le joueur indique l'action qu'il souhaite réaliser grâce à la hotbar) et est observé par le moteur de rendu, qui utilise les coordonnées dynamiques calculées par chaque thread pour dessiner les jardiniers à leurs positions respectives. Enfin, le système de persistance s'appuie sur cet état pour figer la position et la mission en cours lors d'une sauvegarde.

5.2.2 Système d'Actions et Logique Métier

Les actions représentent les tâches unitaires assignées aux jardiniers. L'architecture a été conçue pour être totalement découplée, permettant d'ajouter de nouvelles mécaniques de jeu sans modifier le code de l'entité.

Structures de données et constantes Le système repose sur une hiérarchie de classes dont la racine est la classe abstraite Action. Celle-ci encapsule les données universelles : les coordonnées cibles (targetX, targetY) et le contrat d'exécution perform(Gardener, World). Les actions concrètes (MoveAction, PlowAction, PlantAction, WaterAction, HarvestAction) héritent de ce socle. La file d'attente du jardinier stocke ces objets, permettant une exécution séquentielle. En amont, des classes de type Builder (PlantActionBuilder, etc.) servent de structures de stockage temporaires pour compiler les choix de l'utilisateur (item sélectionné, jardinier choisi) avant l'instanciation finale de l'action.

Algorithme d'exécution et de construction L'algorithme se déroule en deux phases distinctes :

- **Phase de construction** : Le système fait le pont entre l'interface et la logique. Lorsqu'un joueur clique sur un menu, le Builder correspondant accumule les paramètres. Une fois la cible validée sur la grille, la méthode buildAction() instancie l'action concrète et l'injecte dans la file du jardinier.
- **Phase d'exécution** : Le jardinier se déplace d'abord vers la cible via l'algorithme A*. Une fois arrivé, il déclenche perform(). Chaque action possède sa propre logique interne :
 - PlowAction transforme une Tile standard en PlantTile via la méthode sécurisée toPlantTile().
 - PlantAction vérifie la disponibilité d'une ItemSeed dans la grange avant d'appeler plant().
 - HarvestAction arbitre entre deux scénarios : si la plante est MATURE, elle génère un ItemPlant dans l'inventaire ; si elle est MORT, elle invoque clean() pour libérer la case.

Conditions limites et robustesse La robustesse repose sur une vérification systéma-

tique des prérequis avant toute modification du monde. L'algorithme `perform()` vérifie systématiquement la validité de la cible au moment de l'impact : une action de plantation échouera si la case a été modifiée entre-temps par une autre entité. Concernant l'inventaire, le système garantit l'intégrité des ressources en vérifiant que les graines existent toujours au moment de l'action physique, et non seulement au moment du clic. Enfin, la méthode `toPlantTile()` est conçue pour être "atomique" : elle assure le transfert sécurisé des entités présentes sur la case vers la nouvelle instance de tuile pour éviter toute perte de données ou "disparition" de personnage lors du labourage.

Utilisation par les autres fonctionnalités Ce module est le point de convergence entre le Modèle (logique des plantes), la Vue (menus contextuels) et le Contrôleur (clics souris). Il communique étroitement avec la classe `Barn` pour la gestion des stocks et avec le `World` pour la modification de la grille. Le moteur de rendu utilise le type d'action en cours pour adapter les animations des jardiniers, tandis que le `SaveManager` utilise ces objets pour sauvegarder la liste des tâches programmées afin que le jardinier reprenne son travail après un rechargement.

5.2.2.1 L'Action Récolter : Productivité et Maintenance du Terrain

L'action de récolte remplit une double fonction critique au sein de la simulation : elle permet de convertir le temps de croissance des plantes en profit économique tout en assurant la maintenance de la grille par la suppression des cultures ayant péri. Cette polyvalence repose sur un découpage logiciel strict respectant le patron MVC et le patron de conception Builder.

Structures de données et constantes Le processus mobilise trois structures de données interdépendantes. En amont, le `HarvestActionSelector` agit comme un contrôleur d'interface (implémentant `ActionListener`) chargé d'initier la séquence et de définir les règles de sélection. Le `HarvestActionBuilder` sert de conteneur temporaire pour mémoriser l'identité du jardinier sélectionné et les coordonnées cibles avant l'instanciation. Enfin, la classe `HarvestAction` encapsule la logique métier finale. Ce système ne s'appuie pas sur des constantes fixes, mais sur des prédicats dynamiques (`Predicate<Tile>`) qui filtrent en temps réel les cases éligibles (plantes matures ou mortes) selon l'état actuel du Modèle.

Algorithme du cycle de vie de l'action Le déroulement de l'action s'opère selon un algorithme en trois phases distinctes :

- **Phase de Sélection** : Lors de l'activation du bouton dédié, le contrôleur injecte un critère de sélection dans la Vue. Ce filtre exige que la cible soit une `PlantTile` contenant une entité végétale prête à être récoltée (`isHarvestable()`) ou ayant atteint l'état MORT. Le contrôle est alors délégué au module `Display` pour capturer le clic du joueur.
- **Phase de Construction** : Une fois la coordonnée validée, le Builder compile les données et instancie l'objet `HarvestAction`. À cet instant, il invoque `interruptGarde-`

ner() pour purger les tâches obsolètes de l'entité et injecte la commande de récolte en tête de sa file d'attente.

- **Phase d'Exécution** : Lorsque le thread du jardinier atteint la destination, la méthode perform() arbitre entre deux scénarios. Si la plante est à maturité, la méthode harvest() est appelée : elle parcourt l'inventaire de la grange pour incrémenter la quantité de l'item correspondant ou instancier un nouvel ItemPlant si le type est inédit. Si la plante est morte, la méthode clean() est invoquée pour réinitialiser la parcelle, libérant l'espace sans générer de ressources.

Conditions limites et robustesse La sécurité de l'exécution repose sur la gestion du "gap temporel" inhérent au multithreading. Entre le clic de l'utilisateur et l'arrivée physique du jardinier, l'état de la grille peut avoir évolué (par exemple, si un nuisible a consommé la plante). L'algorithme de la méthode perform() réalise donc une vérification dynamique d'intégrité avant toute modification, s'assurant que la tuile est toujours de type PlantTile et que l'objet végétal est présent. Par ailleurs, le système d'inventaire est conçu pour être résilient : l'algorithme de stockage gère l'absence initiale d'items en créant dynamiquement les instances nécessaires, évitant ainsi tout plantage lié à une liste vide ou non initialisée.

Utilisation par les autres fonctionnalités Cette fonctionnalité est un pivot pour la gestion de l'affichage et de l'économie globale. Elle sollicite le composant Display pour basculer la vue en mode sélection contextuelle, fournissant un retour graphique indispensable au joueur. Sur le plan économique, contrairement aux actions de plantation qui pourraient être limitées aux stocks d'un agent, l'action de récolte centralise les ressources en déposant systématiquement les produits dans la grange du monde (world.getBarn()). Ce choix de conception facilite la gestion des richesses et assure la persistance des gains financiers à l'échelle globale de l'exploitation.

5.2.2.2 L'Action Planter : Gestion des Inventaires et Séquençage Logistique

La fonctionnalité de plantation représente l'une des mécaniques les plus complexes de la simulation. Contrairement aux actions unitaires, planter nécessite une vérification croisée des inventaires (Grange et Jardinier), une planification de trajectoire à distance et, si nécessaire, l'orchestration d'une séquence de ravitaillement. Cette complexité est gérée par une architecture hybride mêlant le patron de conception Builder et une gestion de file d'attente de commandes.

Structures de données et constantes La réalisation de cette tâche mobilise plusieurs structures spécialisées. Le PlantActionSelector initialise le processus en compilant un inventaire virtuel des ressources disponibles. L'InventorySelector valide ensuite le choix de l'utilisateur en vérifiant le type de l'objet sélectionné (classe ItemSeed). Le cœur logique de l'opération réside dans le PlantActionBuilder, qui agit comme un cerveau logistique pour séquencer les commandes. Enfin, l'objet PlantAction (héritant de Action) stocke deux paires de coordonnées distinctes : les coordonnées d'exécution et les coordonnées cibles. Les constantes de PlantType définissent les caractéristiques de chaque graine utilisée.

Algorithme de planification et d'exécution Le cycle de vie de cette action se décompose en trois phases algorithmiques :

- **Sélection et Inventaire** : Le système agrège stocks de la Grange et de l'inventaire personnel du jardinier pour chaque espèce végétale. Après la sélection d'une graine, l'utilisateur cible une parcelle de type `isFarmable()`. Ces paramètres sont transmis au Builder.
- **Planification Logistique** : Lors de l'appel à `buildAction()`, le système calcule la tuile adjacente accessible la plus proche de la cible via la méthode `world.findClosestWalkableAdjacent()`. Le Builder arbitre alors entre deux scénarios :
 - **Scénario de possession** : Si le jardinier possède déjà la graine, le Builder génère une séquence simple : un déplacement vers la case adjacente suivi de la `PlantAction`.
 - **Scénario de ravitaillement** : Si l'inventaire personnel est insuffisant, le Builder injecte une séquence complexe comprenant un déplacement vers la grange, une `FetchSeedAction` (récupération de ressource), un nouveau déplacement vers la cible, puis la `PlantAction`.
- **Exécution Métier** : À l'arrivée sur les coordonnées d'exécution, la méthode `perform()` s'active. L'algorithme vérifie dynamiquement si la parcelle est toujours disponible. En cas de succès, la méthode `plant()` de la tuile instancie la plante et déduit une unité de l'inventaire personnel du jardinier.

Conditions limites et robustesse La robustesse du système repose sur une vérification tardive de l'état du monde. Puisque l'environnement est multithreadé et évolue pendant le déplacement du jardinier, la disponibilité de la case est vérifiée à la dernière milliseconde au sein de la méthode `perform()`. Si la case est occupée entre-temps, l'action est avortée de manière sécurisée pour éviter toute incohérence logique. Par ailleurs, l'architecture sépare strictement la position du jardinier de celle de la plante. Cette interaction à distance garantit que l'entité n'occupe jamais physiquement la case cultivée, prévenant ainsi les conflits de superposition d'entités sur la grille.

Utilisation par les autres fonctionnalités Le module de plantation interagit étroitement avec le moteur de rendu pour améliorer l'expérience utilisateur. Le `PlantActionBuilder` communique avec le composant `Display` pour appliquer une surbrillance sur la case cible dès la phase de planification. Pour assurer la propreté visuelle, un mécanisme de callback (via un objet `Runnable`) est attaché à la dernière commande de déplacement : dès que le jardinier atteint sa position d'exécution, le signal est envoyé pour effacer automatiquement la surbrillance. Enfin, cette fonctionnalité sollicite le système d'inventaire global pour la gestion des flux de graines entre la grange et les entités.

5.2.2.3 L'Action Arroser : Maintenance du Cycle de Vie Végétal

L'action d'arrosage est une mécanique vitale assurant la pérennité des cultures face au système d'évaporation de la simulation. À l'instar de la plantation, cette fonctionnalité nécessite une validation dynamique de l'état du Modèle. Elle repose sur le patron de conception Builder pour découpler strictement la phase de sélection par l'utilisateur de la phase d'exécution par l'entité autonome, garantissant ainsi une gestion fluide des interactions dans un environnement multithreadé.

Structures de données et constantes Le système s'articule autour de trois composants principaux. Le WaterActionSelector agit comme un contrôleur (implémentant ActionListener) et sert de point d'entrée pour définir les règles de filtrage sur la grille. Le WaterActionBuilder prend en charge la planification spatiale et l'ordonnancement des tâches. Enfin, la commande WaterAction encapsule les données nécessaires à l'exécution : les coordonnées de positionnement du jardinier (execX, execY) et celles de la cible (waterX, waterY). Contrairement à d'autres actions, l'arrosage ne dépend pas de constantes d'inventaire, mais de l'état interne de la PlantTile et de l'entité Plant qu'elle contient.

Algorithme du cycle de vie de l'action Le flux d'exécution se décompose en trois phases algorithmiques distinctes :

- **Phase de Sélection (Contrôleur) :** À l'activation du mode d'arrosage, le système génère un critère de sélection (Predicate<Tile>) exigeant que la parcelle soit une PlantTile et qu'une instance de plante y soit vivante. Ce filtre autorise l'arrosage à n'importe quel stade de maturité. Le contrôle est alors transféré au module Display pour capturer l'intention du joueur.
- **Phase de Planification Logistique :** Une fois la cible validée, le Builder détermine la position optimale de l'entité. Il interroge le monde via la méthode findClosestWalkableAdjacent() pour identifier la case accessible la plus proche de la cible. La file d'attente du jardinier est alors réinitialisée (via interruptGardener()) pour recevoir une séquence composée d'une MoveAction (déplacement vers la périphérie) et de la WaterAction finale.
- **Phase d'Exécution (Modèle) :** Lorsque le jardinier atteint sa destination, la méthode perform() est invoquée. L'algorithme récupère la tuile ciblée et vérifie dynamiquement son intégrité. Si la plante est toujours présente, la méthode water() est appelée, restaurant le niveau d'eau virtuel de l'entité végétale pour prolonger son cycle de croissance.

Conditions limites et robustesse L'état d'une plante pouvant changer radicalement pendant le trajet du jardinier (mort par dessèchement ou récolte par un autre jardinier), la méthode perform() effectue une vérification d'existence de la plante à la dernière milliseconde. Cette précaution prévient les erreurs de type NullPointerException en cas de disparition de la cible. Par ailleurs, le système prévoit une sécurité de repli : si aucune case

adjacente n'est accessible (cible encerclée), le Builder utilise par défaut les coordonnées de la cible elle-même, permettant au thread de tenter l'action malgré l'obstruction.

Utilisation par les autres fonctionnalités Cette fonctionnalité sollicite intensivement l'algorithme de Pathfinding pour assurer un positionnement cohérent du jardinier sans empiéter sur la zone cultivée. Sur le plan de l'interface, le WaterActionBuilder interagit avec le Display pour générer un retour visuel immédiat. Il applique une surbrillance sur la parcelle visée et injecte une fonction de rappel (Runnable clearHighlight) dans la commande de déplacement. Ce mécanisme garantit que l'indicateur visuel est automatiquement nettoyé dès que l'agent atteint sa position d'exécution, assurant ainsi la clarté de l'interface pour l'utilisateur.

5.2.2.4 L'Action Labourer : Transformation Structurale du Terrain

L'action de labourage représente l'étape fondamentale de la simulation, marquant la transition entre un environnement inerte et un espace de production. Contrairement aux actions de maintenance (arrosage ou récolte), le labourage modifie la structure même du monde en transformant une case de terre basique en une parcelle cultivable.

Structures de données et constantes La réalisation de cette tâche mobilise trois structures interdépendantes. Le PlowActionSelector agit comme le contrôleur de l'interaction initiale, utilisant le critère fonctionnel isPlowable pour filtrer les entrées de l'utilisateur. Le PlowActionBuilder (héritant d'ActionBuilder) assure la conversion des intentions du joueur en un plan d'action logique pour l'agent, incluant la recherche de chemin. Enfin, la commande PlowAction encapsule les données d'exécution finales : les coordonnées de positionnement de l'agent (execX, execY) et les coordonnées de la parcelle à transformer (plowX, plowY). Cette action ne consomme pas de ressources d'inventaire, mais elle est le prérequis indispensable à l'utilisation des constantes de plantation.

Algorithme du cycle de vie de l'action Le processus de transformation se déroule selon un flux algorithmique en trois étapes :

- **Phase de Sélection (Contrôleur et UI) :** Suite à l'activation du mode labourage, le système invoque la méthode switchToSelection() du module Display. Le joueur utilise alors le SelectionController pour cibler une case. Le système garantit la validité du choix en temps réel grâce au prédicat isPlowable ; dès que le clic est validé, les coordonnées sont transmises au Builder.
- **Phase de Planification Logistique :** La méthode buildAction() du Builder calcule la tuile marchable adjacente à la cible pour définir la zone de travail de l'agent. La file d'attente du jardinier est alors réinitialisée (via interruptGardener()) pour recevoir une séquence ordonnée : une MoveAction (déplacement vers la case adjacente) suivie de la PlowAction proprement dite.
- **Phase d'Exécution Métier (Modèle) :** Une fois le jardinier positionné, la méthode perform() est déclenchée. Elle réalise une vérification ultime de l'état de la cible.

En cas de succès, elle sollicite la classe `World` via sa méthode `toPlantTile()`. Cette méthode procède au remplacement définitif de l'instance de la tuile classique par une nouvelle instance de `PlantTile` dans la matrice du monde.

Conditions limites et robustesse La robustesse du système face à la concurrence est assurée par une validation redondante. Bien que l'interface filtre les clics invalides, l'état de la grille peut changer durant le trajet du jardinier (par exemple, si un autre agent a déjà labouré la case). La vérification effectuée au sein de `perform()` protège le système contre ces situations de compétition (Race Conditions) en avortant l'opération de manière sécurisée. Par ailleurs, une sécurité de positionnement est intégrée : si une case est totalement enclavée par des obstacles, le système utilise par défaut les coordonnées de la cible elle-même comme zone d'exécution, laissant l'algorithme A^* gérer l'éventuel blocage du chemin.

Utilisation par les autres fonctionnalités Cette fonctionnalité est unique par son impact sur la grille : contrairement aux autres actions qui modifient les propriétés internes d'un objet, `PlowAction` orchestre une modification structurelle de la matrice du `World`. Elle s'interface également avec le module d'affichage `Display` pour fournir un retour visuel dynamique. Le `PlowActionBuilder` place la case ciblée en surbrillance (highlight) et injecte un rappel dans la commande de déplacement. Ce mécanisme garantit que la surbrillance est retirée avec précision au moment exact où le jardinier commence son travail, assurant une synchronisation parfaite entre la logique et le rendu visuel.

5.2.2.5 L'Action Déplacer : Navigation et Orchestration des Séquences

L'action de déplacement constitue la brique élémentaire de la mobilité au sein de la simulation. Elle peut être instanciée de deux manières : soit directement par le joueur pour repositionner manuellement un agent, soit automatiquement par les autres modules de planification (`PlantActionBuilder`, `WaterActionBuilder`, etc.) pour orchestrer les phases d'approche nécessaires à la réalisation d'une tâche métier. Son architecture repose sur un paradigme asynchrone permettant de synchroniser l'état du Modèle avec celui de la Vue sans créer de couplage fort.

Structures de données et constantes Le système de mouvement s'appuie sur la classe `MoveAction`, qui hérite du socle commun `Action`. Outre les coordonnées de destination finale (`targetX`, `targetY`), sa structure intègre un attribut fondamental : un `Runnable` nommé `onArrival`. Cette structure de données permet de transporter une logique d'exécution optionnelle qui sera déclenchée une fois la destination atteinte. En amont, le `Selection-Controller` assure la capture des entrées du joueur et la conversion des coordonnées-écran en coordonnées-monde, tandis que les différents `ActionBuilder` assure la configuration de l'objet avant son injection dans la file d'attente du jardinier.

Algorithme de navigation et de finalisation Le cycle de vie d'un déplacement suit un flux algorithmique décomposé en quatre phases :

- **Phase de Sélection (Contrôleur)** : Le contrôleur capte l'événement souris et valide la destination via un critère de "marchabilité". Une fois la cible confirmée, les coordonnées sont transmises au Builder.
- **Phase de Planification** : Le Builder instancie la MoveAction. Dans le cas de séquences complexes, il injecte dans le constructeur une fonction de rappel (ex : `clearHighlight`) destinée à nettoyer l'interface graphique à l'arrivée. L'action est alors ajoutée au registre des tâches de l'agent.
- **Phase d'Exécution (Modèle et Navigation)** : Le déplacement physique, case par case, est géré de manière autonome par la boucle de vie du Thread du jardinier (via l'algorithme de pathfinding A*). L'objet MoveAction ne porte pas la logique de trajet, mais sert de marqueur d'objectif.
- **Phase de Finalisation** : Dès que le jardinier coïncide physiquement avec les coordonnées cibles, la méthode `perform()` est déclenchée. Elle exécute alors l'attribut `onArrival.run()`, permettant de notifier la Vue ou de déclencher l'action suivante de la file d'attente.

Conditions limites et robustesse La robustesse du système est garantie par une séparation des responsabilités. La classe MoveAction délègue la gestion des échecs de navigation au Gardener : si une cible devient inaccessible en cours de route (obstruction soudaine), le Thread de l'agent annule l'action avant même l'appel à `perform()`. Par ailleurs, l'exécution du callback `onArrival` est isolée au sein d'un bloc `try/catch`. Cette précaution est vitale pour la stabilité du multithreading : si le code lié à l'interface graphique (souvent plus instable) échoue lors de l'arrivée, l'exception est interceptée sans interrompre le Thread du jardinier, garantissant qu'il puisse traiter les commandes suivantes.

Utilisation par les autres fonctionnalités MoveAction est la composante pivot de la composition des tâches. Elle est systématiquement utilisée par tous les autres Builders du jeu comme "commande d'approche". Cette conception permet de standardiser le comportement des agents : pour arroser, un jardinier reçoit d'abord une MoveAction vers une case adjacente, puis la tâche d'arrosage. Enfin, le mécanisme de Runnable assure un pont de communication élégant avec l'interface utilisateur, permettant de supprimer les marqueurs de sélection ou de mettre à jour des menus contextuels de manière parfaitement synchronisée avec la fin du mouvement physique de l'avatar.

5.2.3 Le Système d'Inventaire : Gestion et Affichage des Ressources

Le système d'inventaire assure la persistance et la manipulation des objets (graines et récoltes) au sein de la simulation. Il est conçu pour séparer strictement la logique de stockage des ressources de leur représentation graphique, permettant ainsi aux agents autonomes de transporter les éléments nécessaires à leurs actions tout en offrant une interface de sélection interactive au joueur.

Structures de données et constantes La gestion des ressources repose sur une hiérarchie de classes spécialisées. La classe abstraite `Item` définit les propriétés universelles d'un objet : son type, sa quantité et son identité visuelle. Elle est déclinée en deux types concrets : `ItemPlant` pour les produits récoltés et `ItemSeed` pour les semences. Le stockage est assuré par la classe `Inventory` qui encapsule une `ArrayList<Item>`. Chaque jardinier possède sa propre instance d'inventaire, lui conférant une autonomie de mouvement et de travail. Côté interface, le `PopupInventory` (View) utilise un `GridLayout` dont les dimensions sont fixées par les constantes `WIDTH` et `HEIGHT`, limitant l'affichage à une grille de 15 emplacements (3x5).

Algorithme d'affichage et d'interaction La génération de l'interface visuelle suit un algorithme de transcription du Modèle vers la Vue :

- **Parcours du Modèle** : Le système itère sur la liste des items stockés dans l'objet `Inventory`. Pour chaque élément présent, il extrait l'icône graphique et le compteur de quantité.
- **Configuration des composants** : Un `JButton` interactif est instancié et configuré avec les données extraites, puis injecté dans la grille.
- **Stabilisation du Layout** : Pour garantir la cohérence visuelle de la grille (`GridLayout`), l'algorithme complète systématiquement les emplacements vides par des composants invisibles (`JLabel`) jusqu'à atteindre le nombre total défini par les constantes de dimension.
- **Liaison événementielle** : Chaque bouton est relié à un contrôleur chargé de transmettre l'item sélectionné au système de construction d'actions (`ActionBuilder`) via un médiateur nommé `InventorySelector`.

Conditions limites et robustesse La robustesse du système d'affichage est contrainte par ses limites géométriques. L'interface est strictement bornée à 15 cases ; par conséquent, si un inventaire dépasse cette capacité, l'algorithme de rendu ignore les éléments excédentaires pour éviter tout débordement graphique. Pour la gestion des stocks, les méthodes `addItem` et `removeItem` de la classe `Inventory` assurent l'intégrité des données : elles vérifient systématiquement la présence d'un type d'objet avant toute soustraction pour prévenir les quantités négatives. Enfin, la présence de composants de remplissage invisibles garantit que l'interface ne s'effondre pas visuellement lorsque l'inventaire est partiellement vide.

Utilisation par les autres fonctionnalités L'inventaire est le moteur de la logique métier lors de l'exécution des tâches. Lorsqu'un jardinier exécute la méthode `perform()` d'une action (comme planter ou récolter), il interagit directement avec son instance d'inventaire pour consommer une graine ou stocker un produit. Par ailleurs, il constitue une étape pivot dans le système de sélection : le `PopupInventory` sert de passerelle entre le désir du joueur et la planification logistique, permettant au `PlantActionBuilder` de savoir précisément quelle ressource doit être mobilisée pour la mission à venir.

5.2.4 Gestion Concurrente de Plusieurs Jardiniers

L'une des plus grandes complexités techniques de la simulation réside dans l'abandon du modèle classique du "personnage unique" au profit d'un système multi-agents. Le jeu permet de contrôler simultanément deux jardiniers indépendants sur la même grille. Ce choix de game design, qui permet au joueur d'optimiser son temps en parallélisant les tâches, transforme radicalement l'architecture logicielle : il ne s'agit plus seulement d'animer un sprite, mais de gérer un écosystème hautement concurrent.

Paradigme Multi-Thread et Indépendance Logique Afin de garantir que les actions d'un jardinier ne bloquent jamais celles de l'autre, chaque agent est encapsulé dans son propre Thread.

- **Instanciation** : Lors de l'initialisation de la partie, la classe World crée et stocke les entités dans une liste (List<Gardener>). Chacune de ces instances possède sa propre position (x, y), son propre état (currentState) et, surtout, sa propre file d'attente d'actions (Queue<Action> actionQueue).
- **Exécution asynchrone** : Au démarrage, la méthode start() est appelée sur chaque jardinier, lançant deux threads distincts nommés par exemple GardenerThread-1 et GardenerThread-2. À partir de cet instant, les deux boucles run() s'exécutent en parallèle. Si le Jardinier 1 entame un long calcul de chemin (Pathfinding A*) ou marque une pause pour récolter (Thread.sleep()), le Thread du Jardinier 2 continue de s'exécuter à pleine vitesse. Le moteur de rendu (Rendering), s'exécutant sur un autre thread, se contente de lire les coordonnées de ces deux agents pour les dessiner dynamiquement.

Mécanique de Sélection et Contextualisation des Ordres Avoir deux agents autonomes pose un défi majeur pour l'Interface Utilisateur : lorsqu'un joueur clique sur "Planter", à quel jardinier cet ordre s'adresse-t-il ? Pour résoudre ce problème, l'architecture introduit la notion de "Jardinier Actif" (ou sélectionné).

- **Ciblage visuel** : En mode Global, le joueur peut cliquer sur l'un des deux personnages. Le contrôleur détecte l'intersection entre les coordonnées du clic et celles d'un Gardener.
- **Mise à jour du contexte** : Le jardinier cliqué devient la référence active transmise aux contrôleurs d'interface (comme PlantActionSelector ou HarvestActionSelector).
- **Construction spécifique** : Lorsque le joueur valide une commande, le patron Builder (ex : PlantActionBuilder) reçoit explicitement ce jardinier en paramètre. Ainsi, la méthode buildAction() injectera la commande finale exclusivement dans la file actionQueue du bon thread. Le joueur peut donc sélectionner le Jardinier 1, lui ordonner de labourer une ligne entière, puis immédiatement sélectionner le Jardinier 2 et l'envoyer récolter à l'autre bout de la carte.

Prévention des Conflits Concurrents Le fait que deux threads agissent sur le même espace mémoire partagé (la matrice de tuiles du World) expose le programme à des conditions de concurrence critiques. Que se passe-t-il si le joueur donne l'ordre aux deux jardiniers de récolter exactement la même plante au même moment ? L'architecture prévient les crashes et les duplications de ressources grâce à une stratégie d'évaluation paresseuse et de sécurisation logique au sein du Modèle :

- **Découplage temporel** : Le moment où l'ordre est donné (clic du joueur) et le moment où l'ordre est exécuté (arrivée du jardinier sur la case) sont séparés par un laps de temps asynchrone (le déplacement).
- **Validation tardive (perform)** : Aucune vérification faite au moment du clic n'est considérée comme acquise. Lors de la phase de planification, l'interface vérifie bien que la plante est récoltable. Cependant, c'est uniquement une fois arrivé sur la case que le Thread du jardinier exécute la méthode `perform()`.

Le scénario de conflit résolu : Si le Jardinier 1 et le Jardinier 2 arrivent sur la même case pour récolter la même tomate :

- Le Jardinier 1 (arrivé 1 milliseconde plus tôt) exécute `perform()`, vérifie que la plante est là, la récolte, ajoute le produit dans la grange, et détruit l'objet Plant sur la tuile (référence passée à null).
- Le Jardinier 2 exécute son `perform()` juste après. Sa vérification conditionnelle (`if (plant != null && plant.getState() == MATURE)`) retourne false. Il avorte donc son action silencieusement et passe à la tâche suivante de sa file d'attente. Aucun doublon d'objet n'est créé dans l'inventaire, et aucune exception `NullPointerException` ne fait planter le jeu.

Résolution du Bug Visuel Multi-Agents : Le Compteur de Surbrillances Gérer la logique métier (le Modèle) de deux jardiniers est une chose, mais gérer le retour visuel (la Vue) en est une autre. Un problème complexe de "scintillement" ou de disparition prématurée des indicateurs visuels (les highlights jaunes sur les cases ciblées) a dû être résolu techniquement.

Le problème initial : Lorsqu'une action est assignée, le Builder demande à la classe Global de mettre la case cible en surbrillance. Il attache également une fonction de nettoyage au `MoveAction` du jardinier, lui ordonnant de supprimer cette surbrillance une fois arrivé. Or, si le joueur ordonne au Jardinier 1 d'arroser la case (5,5), puis ordonne au Jardinier 2 d'arroser également la case (5,5), la case s'allume en jaune. Si le Jardinier 1 arrive en premier, son callback éteint la surbrillance de la case (5,5). Le Jardinier 2 se retrouve alors à marcher vers une cible qui n'est plus en surbrillance, ce qui casse le retour visuel pour le joueur.

La solution technique implémentée : Au lieu de stocker un simple booléen ("case allumée

ou éteinte"), la classe Global utilise une table de hachage sophistiquée de type `Map<Point, Integer>`.

- **Incrémentation (`setHighlight`)** : Quand le Jardinier 1 vise la case (5,5), la méthode cherche le point dans la Map. S'il n'existe pas, elle l'ajoute avec la valeur 1. Quand le Jardinier 2 vise la même case, la méthode met la valeur à 2. Le moteur de rendu voit que la valeur est > 0 , il dessine le carré jaune.
- **Décrémentation sécurisée (`clearHighlight`)** : Quand le Jardinier 1 arrive, son callback demande l'effacement. Au lieu de supprimer le marqueur, la méthode décrémente la valeur de 2 à 1. Le carré jaune reste donc affiché à l'écran !
- **Nettoyage final** : Ce n'est que lorsque le Jardinier 2 arrive à son tour que le compteur passe de 1 à 0, moment où la case est définitivement retirée de la Map.

Ce système, englobé dans des blocs synchronized pour garantir la sécurité d'accès entre les multiples threads des jardiniers et le thread de la caméra, garantit une Interface Utilisateur parfaite et robuste, quel que soit le niveau de frénésie ou le nombre d'agents actifs à l'écran.

Synergie et Division du Travail La présence de plusieurs jardiniers nécessite une gestion intelligente des ressources pour éviter les impasses logistiques.

- **Ressources Globales (`Grange`)** : Les produits récoltés et l'argent sont centralisés. Peu importe quel jardinier effectue la récolte via `HarvestAction`, les fruits de son travail sont envoyés vers l'instance unique de Barn du monde.
- **Logistique d'approche (`FetchSeedAction`)** : À l'inverse, l'action de plantation requiert un objet physique en possession du jardinier. Lors de l'assignation d'une tâche de plantation à un agent, le `PlantActionBuilder` analyse son inventaire local. Si le Jardinier 2 est sélectionné pour planter une carotte mais qu'il n'a pas la graine sur lui, le système d'orchestration génère automatiquement une file d'actions de ravitaillement : il calcule le chemin vers la Grange, ajoute une action pour retirer la graine du stock global, puis calcule le chemin vers la parcelle cible pour exécuter la plantation.

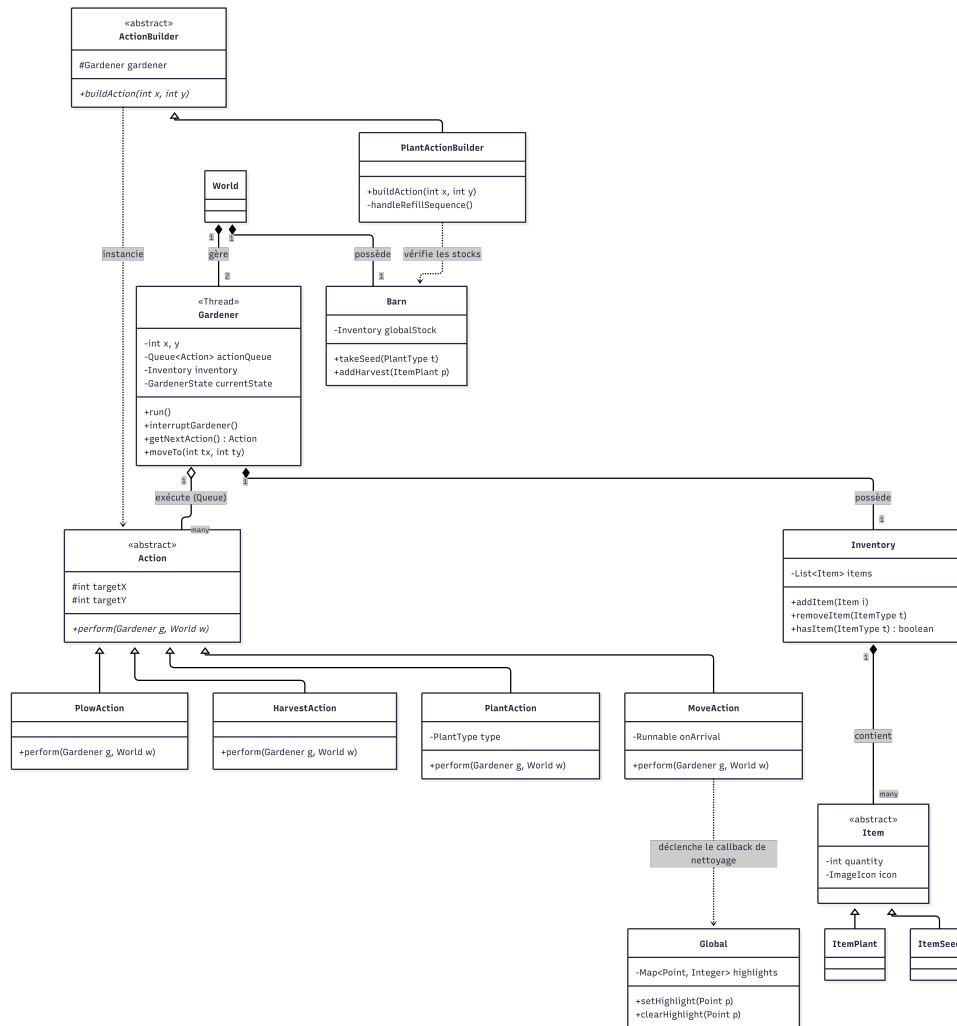


Figure 4 – Diagramme de classe de la fonctionnalité 5.2

5.3 Cycle Végétal & Eau

Cette section détaille l'implémentation de la simulation biologique au sein de la grille. L'objectif de ce module est de créer un écosystème autonome où le passage du temps, cadencé par des ticks, influence l'état des entités végétales et les ressources environnementales sans nécessiter d'intervention systématique du joueur.

5.3.1 Cycle de Vie des Plantes

Le cycle de vie gère l'évolution temporelle de la plante, depuis l'étape de semence jusqu'à sa pleine maturité ou sa décomposition finale. Cette mécanique est modélisée par une machine à états finis intégrée à chaque instance de la classe Plant.

Structures de données et constantes Chaque plante est définie par deux énumérations complémentaires : `PlantState`, qui répertorie les états physiologiques (GRAINE, POUSSÉ, MATURE, POURRIE, MORT), et `PlantType`, qui centralise les métadonnées immuables de l'espèce (durée de croissance, besoins hydriques, valeur marchande). L'état

dynamique est maintenu par un compteur d'âge et un indicateur booléen (`hasFertilizer`) pour la gestion des bonus. Le modèle s'appuie sur des constantes temporelles strictes, notamment `OVERWATER_THRESHOLD` et `TIME_BEFORE_DEATH`, qui fixent la quantité maximale d'eau acceptable sans que la plante pourrisse et le délai de mort par assèchement respectivement.

Algorithme d'évolution temporelle À chaque itération du moteur de simulation (méthode `tick()`), un algorithme séquentiel est appliqué à chaque entité vivante. Le système vérifie d'abord l'intégrité vitale de la plante : si celle-ci est déjà dans un état terminal (`MORT` ou `POURRIE`), le cycle de croissance s'interrompt. Dans le cas contraire, si les conditions hydriques sont réunies (`currentWater > 0`), le système incrémente l'âge de la plante. Cette progression est pondérée par la présence d'engrais : l'âge augmente de deux unités si le bonus est actif, contre une seule en temps normal. L'algorithme procède ensuite aux transitions d'états en comparant l'âge accumulé aux seuils définis par le `PlantType` : un passage à l'état de `POUSSE` est déclenché à 33 % de la durée totale, tandis que la maturité (`MATURE`) est atteinte à 100 %.

Conditions limites et robustesse La stabilité de la simulation repose sur une progression d'état strictement monotone : l'algorithme garantit qu'une plante ne peut jamais régresser vers un stade antérieur de son développement. La variable d'âge est bornée implicitement par les transitions d'états, empêchant tout dépassement de mémoire ou valeur incohérente. Concernant les interactions, une sécurité logique interdit l'application d'engrais sur une plante ayant déjà atteint la maturité ou ayant péri, limitant l'utilisation des bonus aux phases actives de croissance. Cette restriction prévient les gaspillages de ressources et les comportements aberrants du modèle biologique.

Utilisation par les autres fonctionnalités Ce cycle de vie est la source de données primaire pour le moteur de rendu. La couche visuelle (`Display`) interroge en permanence les méthodes `getState()` et `getType()` pour sélectionner le sprite adéquat dans la bibliothèque de textures (passant par exemple d'une texture de bourgeon à celle d'un fruit mûr). Par ailleurs, cet état est scruté par les entités autonomes : les nuisibles comme les corbeaux utilisent l'état de la plante pour identifier leurs cibles, privilégiant les stades de croissance avancés pour leurs interactions de prédation. Le mode sélection demande aussi de sélectionner des plantes dans un certain état (`MATURE` par exemple pour récolter).

5.3.2 Consommation d'eau et Survie : Gestion de l'Humidité

Cette fonctionnalité régit l'aspect critique de la simulation : la dépendance vitale des cultures vis-à-vis des ressources hydriques. Elle simule l'assèchement progressif du sol et la capacité de résilience des plantes face à la sécheresse, introduisant une gestion du risque et une nécessité de vigilance constante pour le joueur.

Structures de données et constantes La gestion de l'humidité est traitée comme une ressource scalaire flottante rattachée à chaque instance de `Plant`. La variable `currentWaterLevel` (comprise entre 0.0 et `MAX_WATER_LEVEL`) représente le niveau

d'humidité actuel au pied de la plante. En complément, un compteur entier `ticksWithoutWater` mémorise le temps passé en état de stress hydrique total. Le modèle est calibré par plusieurs constantes environnementales : `MAX_WATER_LEVEL` définit la capacité de rétention maximale d'une parcelle, tandis que `EVAPORATION_RATE` et `waterConsumption` (propre à chaque `PlantType`) déterminent la vitesse de perte passive et active de l'eau. Enfin, `TIME_BEFORE_DEATH` fixe le seuil de tolérance de la plante avant que la déshydratation ne devienne fatale.

Algorithme de simulation hydrique Le moteur de simulation exécute un algorithme de mise à jour à chaque cycle (`tick()`) selon une logique à trois étapes :

- **Phase de Consommation et Évaporation** : Le système calcule la perte d'eau totale en additionnant la consommation biologique de la plante et l'évaporation naturelle du sol. Si `currentWaterLevel` est supérieur à zéro, cette valeur est déduite et le compteur de stress `ticksWithoutWater` est systématiquement réinitialisé, signifiant que la plante puise dans ses réserves.
- **Phase d'Assèchement** : Un mécanisme de bornage s'assure que le niveau d'eau ne devienne jamais négatif. Si le retrait calculé fait descendre la réserve sous le seuil critique, la valeur est forcée à zéro.
- **Phase de Survie et Déshydratation** : Si le niveau d'eau atteint zéro, l'algorithme cesse d'incrémenter l'âge de la plante (arrêt de la croissance) et commence à incrémenter `ticksWithoutWater`. Dès que ce compteur franchit la limite imposée par `TIME_BEFORE_DEATH`, l'automate de la plante bascule définitivement vers l'état MORT.

Conditions limites et robustesse La robustesse de ce module repose sur l'irréversibilité des états terminaux. Une fois que le seuil de survie est dépassé, la mort par déshydratation est définitive : l'algorithme de survie ignore toute nouvelle tentative d'arrosage, empêchant ainsi la "résurrection" d'une culture flétrie. Par ailleurs, le système gère les surplus d'irrigation de manière sécurisée ; lors de l'appel à la méthode de recharge, tout apport dépassant `MAX_WATER_LEVEL` est ignoré par le modèle, simulant la saturation du sol et évitant des valeurs de réserve incohérentes qui pourraient fausser les calculs de croissance ultérieurs.

Utilisation par les autres fonctionnalités Ce système est un indicateur clé pour la couche visuelle. La classe `Display` utilise la méthode `isIrrigated()` pour modifier dynamiquement l'apparence du sol sur la grille : une valeur d'eau positive déclenche l'affichage d'une texture de terre foncée (humide), tandis qu'un niveau nul affiche une terre claire (sèche), offrant un retour visuel immédiat au joueur. De plus, ce module est la cible directe du contrôleur d'actions : l'action `WaterAction` (Arrosage) invoque la méthode `water(amount)` pour réinjecter des ressources dans le modèle, réactivant ainsi immédiatement le cycle de croissance si la plante n'a pas encore atteint le stade de mort définitive.

5.3.3 Interactions Agricoles : Lien entre Grille et Logique Biologique

Cette sous-fonctionnalité assure l'interface entre la structure spatiale du jeu (Tile) et l'entité biologique (Plant). Elle garantit l'intégrité des règles de placement et de gestion des cultures, en agissant comme une couche transactionnelle qui valide chaque manipulation agricole avant de modifier l'état du Modèle.

Structures de données et constantes La réalisation technique de ce module s'appuie sur le principe de spécialisation par héritage. La classe PlantTile étend la classe générique Tile pour lui adjoindre des propriétés spécifiques au jardinage. La structure de données centrale est une référence vers un objet Plant, intégrée au sein de chaque PlantTile. Cette référence est "nullable" : une valeur nulle indique une parcelle labourée mais disponible, tandis qu'une instance non nulle signifie qu'une culture est en cours. Cette architecture permet de décorer la grille de fonctionnalités avancées (méthodes de plantation, d'arrosage et de récolte) sans alourdir les classes gérant les chemins ou les décors inertes.

Algorithmes de manipulation transactionnelle Les interactions reposent sur des algorithmes de vérification et d'exécution déclenchés par les actions du joueur ou des agents :

- **Algorithme de Plantation** : Lorsqu'un type de plante est soumis, le système effectue une vérification d'occupation. Si la référence plant est nulle, l'algorithme procède à l'instanciation d'un nouvel objet Plant selon le type requis. Dans le cas contraire, l'opération échoue pour prévenir la superposition de cultures.
- **Algorithme de Récolte** : Ce processus valide deux conditions successives : la présence effective d'une entité et son stade de maturité (state == MATURE). Si ces critères sont remplis, l'algorithme extrait la valeur marchande définie dans le PlantType pour l'ajouter aux ressources globales, puis réinitialise la référence à null, libérant ainsi la parcelle.
- **Algorithme de Nettoyage** : Spécifiquement conçu pour la maintenance, cet algorithme identifie les entités dont l'état est MORT ou POURRIE. Son exécution entraîne la suppression de l'objet plante sans contrepartie financière, permettant de recycler l'espace de culture.

Conditions limites et robustesse La sécurité du système repose sur le typage fort et l'exclusivité spatiale. L'architecture garantit techniquement qu'une case ne peut contenir qu'une seule plante à la fois, interdisant toute fuite mémoire ou conflit d'entités sur une même coordonnée. De plus, la robustesse est assurée par le polymorphisme : les méthodes de plantation n'existant que dans la classe PlantTile, il est impossible d'initier une action agricole sur une Tile classique (comme un sentier ou un réservoir). Cette barrière logicielle prévient les erreurs d'exécution lors des clics imprécis de l'utilisateur ou des erreurs de trajectoire des agents.

Utilisation par les autres fonctionnalités Ce module agit comme un point de conver-

gence pour le contrôle et la simulation. Le contrôleur global (GlobalController) utilise ces méthodes pour traduire les interactions souris en modifications du terrain selon l'outil sélectionné par le joueur. Parallèlement, la classe World s'appuie sur cette structure lors de ses itérations périodiques : elle parcourt la grille et appelle la méthode tick() de chaque case. La PlantTile délègue alors cet appel à sa plante interne, assurant ainsi que chaque entité végétale progresse dans son cycle de vie de manière synchronisée avec le reste de l'univers simulé.

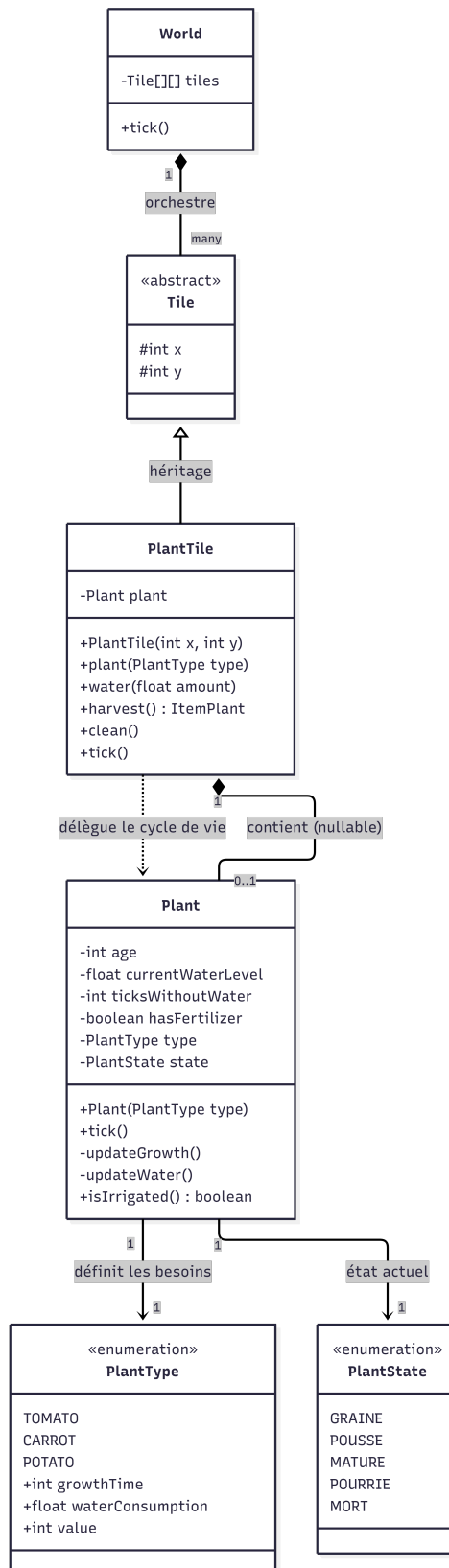


Figure 5 – Diagramme de classe de la fonctionnalité 5.3

5.4 Entités et Déplacements (Pathfinding)

Cette section détaille l'implémentation de la logique de navigation partagée par l'ensemble des entités mobiles de la simulation, qu'il s'agisse des jardiniers ou des animaux nuisibles. L'objectif est de doter ces entités d'une intelligence spatiale leur permettant de rallier une destination de manière optimale tout en contournant dynamiquement les obstacles présents sur la grille.

5.4.1 Algorithme de Recherche de Chemin (A*)

Pour résoudre le problème du plus court chemin dans un environnement discret, nous avons implémenté l'algorithme A-étoile (A*). Ce choix se justifie par son efficacité et sa capacité à utiliser une heuristique pour guider la recherche, limitant ainsi le nombre de cases explorées par rapport à un algorithme de type Dijkstra classique.

Structures de données et constantes L'algorithme est centralisé au sein de la classe mère Entity pour favoriser la réutilisation du code. Afin de garantir la sécurité des données dans un contexte multithreadé (plusieurs entités pouvant calculer des trajectoires simultanément), toutes les structures de calcul sont instanciées localement à la méthode de recherche. Le système s'appuie sur une classe interne privée nommée Node, représentant une cellule évaluée. Chaque nœud stocke ses coordonnées, une référence vers son "parent" pour la reconstruction du chemin, ainsi que trois coûts : le gCost (coût réel depuis le départ), le hCost (estimation heuristique vers l'arrivée) et le fCost (somme des deux précédents). Le processus de recherche utilise une openList implémentée via une simple ArrayList<Node>. Une closedList, modélisée par une matrice bidimensionnelle de booléens (boolean[][]), permet de marquer instantanément les cases déjà traitées et d'optimiser les temps d'accès.

Algorithme de calcul de trajectoire Lorsqu'une entité sollicite un déplacement, la méthode pathFinding() exécute un algorithme structuré en quatre étapes :

- **Initialisation et Vérification** : Le système valide d'abord que la cible est comprise dans les limites du World. La vérification de franchissement n'est pas strictement booléenne : si la case est normalement obstruée (!isWalkable()), l'algorithme vérifie s'il s'agit d'une porte (isGate()). Si c'est le cas, et que l'entité effectuant la recherche est un Jardinier, l'accès est exceptionnellement autorisé. Le nœud de départ est ensuite injecté dans l' openList.
- **Boucle de Recherche** : Tant que l' openList n'est pas vide, l'algorithme sélectionne le nœud possédant le plus bas fCost. Si plusieurs nœuds ont le même coût total, l'heuristique hCost (calculée via la Distance de Manhattan : $|x_1 - x_2| + |y_1 - y_2|$) sert de critère de départage. Ce nœud devient le "nœud courant".
- **Exploration des Voisins** : Le système examine les quatre voisins orthogonaux. La même logique asymétrique de franchissement des portes s'applique ici pour

déterminer si le voisin est traversable. Les voisins hors limites ou déjà présents dans la `closedList` sont ignorés. Pour les autres, l'algorithme calcule les nouveaux coûts et met à jour leurs références parentales si un chemin plus court est découvert.

- **Reconstruction du Chemin :** Si le nœud courant coïncide avec la cible, l'algorithme remonte la chaîne des parents jusqu'à l'origine. La liste de `Tile` résultante est inversée pour fournir une séquence ordonnée de déplacements de l'entité vers son objectif.

Conditions limites et robustesse La robustesse de la navigation repose sur la gestion des contraintes spatiales et dynamiques. L'algorithme ignore strictement les déplacements en diagonale pour respecter la topologie en croix de la grille de jeu. En cas d'absence totale de chemin (cible encerclée), la boucle se termine naturellement et retourne une valeur nulle, permettant à l'entité d'annuler son action proprement. De plus, comme le critère `isWalkable()` est interrogé en temps réel, l'algorithme s'adapte instantanément aux modifications structurelles du monde, comme l'ajout de barrières ou de bâtiments par le joueur, sans nécessiter de recalcul global de la carte de navigation.

Utilisation par les autres fonctionnalités L'algorithme A^* est le moteur décisionnel des `Threads` régissant les jardiniers et les nuisibles. Lorsqu'une entité passe à l'état "En déplacement", il consomme la liste de cases générée pour mettre à jour ses coordonnées de manière séquentielle, en intégrant des pauses via `Thread.sleep()` pour simuler la vitesse de marche. Enfin, la couche visuelle (`Display`) accède à ces coordonnées dynamiques à chaque cycle de rendu, assurant que le sprite de l'entité suit parfaitement la trajectoire calculée, offrant ainsi une animation fluide et cohérente sur l'interface graphique.

5.4.2 Gestion des Ennemis et Nuisibles : Entités Hostiles et Concurrence

L'une des dynamiques principales de la simulation repose sur l'interaction avec des entités hostiles autonomes. Pour garantir l'indépendance de leurs actions par rapport aux interventions du joueur et assurer la fluidité du rendu, ces nuisibles s'appuient sur une architecture multithreadée robuste.

Structures de données et Architecture L'ensemble des entités mobiles hérite de la classe abstraite `Entity`, qui centralise les attributs spatiaux (`x`, `y`), la direction du regard (`facingDirection`) et l'algorithme de pathfinding A^* . Les classes concrètes `Chicken` et `Crow` implémentent l'interface `Runnable`, isolant ainsi leur comportement dans un fil d'exécution dédié.

- **La Poule :** Utilise une énumération de contrôle (`IDLE`, `RUNNING`, `EATING`, `FLEEING`) et un compteur de satiété (`plantsEaten`) limité à 3 unités.
- **Le Corbeau :** Intègre un compteur d'ennui et une logique de détection de zone d'influence pour l'épouvantail.
- **Sécurité Concurrente :** Les instances sont stockées dans une `CopyOnWriteAr-`

rayList. Cette structure est vitale pour permettre au thread de rendu (Rendering) d'itérer sur les entités pendant que les Spawners en insèrent de nouvelles, évitant ainsi toute ConcurrentModificationException.

Algorithmes comportementaux Le cycle de vie des nuisibles suit des logiques de décision distinctes selon leur nature :

- **La Poule** : Son algorithme de ciblage (findNearestPlant) scanne la grille pour identifier la PlantTile cultivée la plus proche. Elle utilise l'algorithme A* pour naviguer, en intégrant une vérification anticollision (hasChicken()) pour éviter la superposition des entités. Une fois sur cible, elle entre dans une phase de consommation de 3 secondes avant de détruire la plante via plant.destroyByEnemy(). Entre deux actions, elle exécute une "balade aléatoire" (wanderRandomly) pour simuler un comportement naturel.
- **Le Corbeau** : Contrairement à la poule, le corbeau ignore les contraintes de franchissement des tuiles. Sa méthode getFlightPath() calcule un vecteur géométrique rectiligne vers sa cible, lui permettant de survoler les obstacles (clôtures, bâtiments). Sa logique de survie inclut une vérification dynamique (isNearScarecrow) à chaque case survolée. S'il pénètre dans un rayon de 3 cases autour d'un épouvantail, son état bascule en FLEEING, le forçant à quitter la carte.
- **Spawners** : Des threads superviseurs (ChickenSpawner et CrowSpawner) gèrent l'apparition des ennemis à intervalles réguliers (SPAWN_INTERVAL de 20s). Ils sélectionnent aléatoirement un bord de la matrice pour l'instanciation (spawnAtEdge) et respectent des quotas de population (MAX_CHICKENS / MAX_CROWS) pour préserver l'équilibre et les ressources CPU.

Conditions limites et Robustesse Le système impose des barrières strictes pour maintenir la cohérence du gameplay. Une poule rassasiée (3 plantes consommées) ou recevant un clic du joueur déclenche immédiatement sa méthode flee(), l'orientant vers la sortie la plus proche. Pour le corbeau, l'apparition est conditionnée par la progression du joueur (level >= 4), garantissant que les outils de défense nécessaires (épouvantails) soient techniquement accessibles. La robustesse est également assurée par l'utilisation de Thread.sleep() et de drapeaux de contrôle (isRunning), permettant un arrêt propre des processus lors de la fermeture de la session de jeu sans laisser de "threads fantômes".

Utilisation par les autres fonctionnalités Ces entités interagissent dynamiquement avec la grille via les méthodes addEntity() et removeEntity() de la classe Tile, assurant que leur position est connue du moteur de jeu à chaque instant. Elles sollicitent également le SoundManager pour fournir un feedback auditif (cris lors de l'apparition ou de la fuite), renforçant l'immersion. Enfin, elles sont les cibles prioritaires du moteur de rendu, qui adapte les sprites et les animations en fonction de l'état interne (currentState) de chaque thread ennemi.

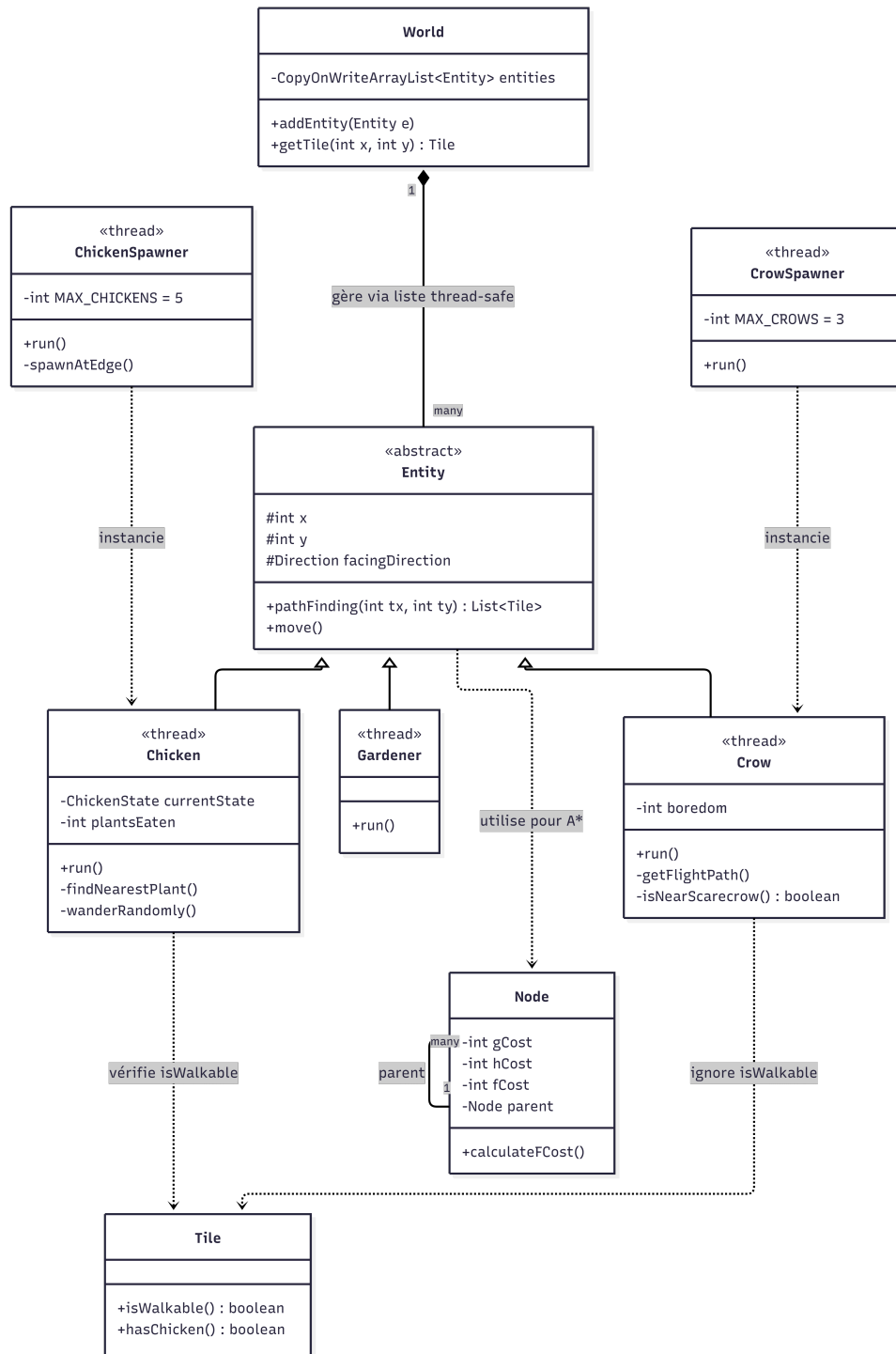


Figure 6 – Diagramme de la fonctionnalité 5.4

5.5 Constructions et Bâtiments

Cette fonctionnalité permet au joueur de personnaliser son exploitation, de structurer l'espace et d'accéder à de nouvelles mécaniques de jeu. L'architecture repose sur une classe mère abstraite **Building** et un gestionnaire central, le **BuildingManager**, qui assure l'intégrité de la grille lors des modifications structurelles.



Figure 7 – Screen du panel de construction

5.5.1 Le BuildingManager : Contrôle et Placement

Le BuildingManager agit comme le contrôleur principal pour toutes les opérations de pose et de suppression. Il arbitre les interactions entre l'interface utilisateur et la base de données spatiale du monde.

Structures de données et constantes Le manager gère l'état de l'interface via des drapeaux booléens (isPlacing, isDeletionMode). Lorsqu'un bâtiment est sélectionné, il instancie un ghostBuilding. Le système s'appuie sur l'énumération PlacementRule intégrée à la classe Building pour définir les contraintes de pose : NORMAL_ONLY (herbe uniquement), PLANTABLE_ONLY (terres cultivables) ou ANY. Les coordonnées du fantôme (ghostX, ghostY) sont stockées en format "grille" après conversion des coordonnées pixels de la souris.

Algorithme de placement et de prévisualisation L'algorithme de pose suit un cycle de validation en temps réel :

- **Projection** : À chaque mouvement de souris, le système convertit la position du curseur en coordonnées de grille en tenant compte du décalage de la caméra.
- **Validation (canPlace)** : La méthode vérifie trois conditions critiques : l'inclusion dans les limites du World, l'absence de bâtiment préexistant via world.getBuildingAt(x, y) et le respect de la PlacementRule par rapport au TileType de la case.
- **Transaction** : Lors du clic de confirmation, le bâtiment est injecté dans la liste du monde. La méthode addBuilding du World procède alors à l'altération physique de la tuile en appelant setWalkable(false) et setPlowable(false) selon les propriétés de l'objet posé.
- **Suppression** : En mode destruction, l'algorithme identifie le bâtiment ciblé, le retire

du registre et restaure les propriétés d'origine de la tuile (réactivation de la marche et du labourage).

Conditions limites et robustesse Le système prévient toute superposition incohérente en bloquant la validation si une seule case de l'empreinte du bâtiment est déjà occupée ou si le type de sol est incompatible. La robustesse est assurée par le rendu visuel : le manager transmet un état de validité à la vue, qui dessine un rectangle translucide bleu ou rouge, offrant un garde-fou visuel avant que le joueur ne dépense ses ressources.

Utilisation par les autres fonctionnalités Le BuildingManager est le point d'entrée pour toutes les modifications durables de la carte. Il communique avec le BuildingSidePanel pour la sélection des types et avec la classe Global pour l'affichage des prévisualisations.

5.5.2 Catalogue des Bâtiments et Infrastructures : Fonctionnalités et Effets

Le catalogue de constructions offre au joueur des outils pour automatiser sa production, protéger ses cultures et personnaliser son exploitation. Chaque structure est définie par des attributs physiques (width, height), économiques (buyPrice) et de progression (levelRequirement), assurant une montée en puissance cohérente du gameplay.

Structures de données et constantes Tous les bâtiments héritent de la classe abstraite Building et sont stockés dans une List<Building> centralisée au sein de la classe World.

- **Bâtiments Fonctionnels** : La BarnBuilding occupe une empreinte de 3x3 cases et est soumise à une contrainte de multiplicité de 1. L'Arroseur (niveau 3) et l'Épouvantail (niveau 5) possèdent des paramètres d'influence spatiale : l'épouvantail définit une zone de protection de 7x7 cases (rayon de 3).
- **Systèmes de Passage** : Les portes (GateFace, GateSide) intègrent une logique de filtrage par type d'entité.
- **Éléments de Décor** : Les objets tels que les rochers (Rock1), les arbres (Tree1) ou les enseignes thématiques sont gérés comme des bâtiments passifs occupant une seule tuile.

Algorithmes d'effets et de cycles Le dynamisme des bâtiments repose sur l'algorithme de mise à jour du World. À chaque cycle de simulation (tick()), le monde itère sur sa liste de bâtiments pour appeler la méthode applyEffect(World world). Ce mécanisme permet d'exécuter des logiques asynchrones :

- **Système d'Arrosage Automatisé** : Lors de son applyEffect, l'arroseur identifie les PlantTile adjacentes et incrémente leur currentWaterLevel, simulant une irrigation permanente.
- **Dissuasion des Corbeaux** : L'algorithme de l'épouvantail scanne la liste des entités présentes dans son rayon d'influence. Si une instance de Crow est détectée, le

bâtiment modifie directement l'état de l'entité en FLEEING, déclenchant sa fuite immédiate.

- **Navigation au travers de Portes :** L'algorithme A* utilise une condition spécifique pour les portes : la propriété isWalkable est évaluée dynamiquement. Elle retourne true si l'entité qui tente de traverser est une instance de Gardener, mais demeure false pour les nuisibles (Chicken), créant ainsi un sas de sécurité sélectif.

Conditions limites et robustesse La robustesse du système est assurée par le contrôle des empreintes au sol. Pour les bâtiments multi-cases comme la Grange (3x3), l'algorithme canPlace du BuildingManager vérifie la disponibilité de l'intégralité des 9 tuiles cibles avant de valider la transaction. De plus, le système de progression (levelRequirement) empêche l'instanciation précoce d'infrastructures complexes, garantissant que le joueur possède les ressources et les connaissances nécessaires pour les exploiter. Enfin, l'utilisation d'une liste de bâtiments découplée de la matrice des tuiles permet de supprimer ou de déplacer des structures sans risquer de corrompre les données fondamentales du terrain.

Utilisation par les autres fonctionnalités Ce catalogue est directement sollicité par le ShopPanel pour l'affichage des coûts et des conditions de déblocage. Le moteur de rendu (Rendering) interroge chaque bâtiment pour obtenir son sprite. Enfin, la sauvegarde des données (SaveManager) s'appuie sur la liste des bâtiments du World pour reconstruire l'intégralité des infrastructures lors du chargement d'une partie.

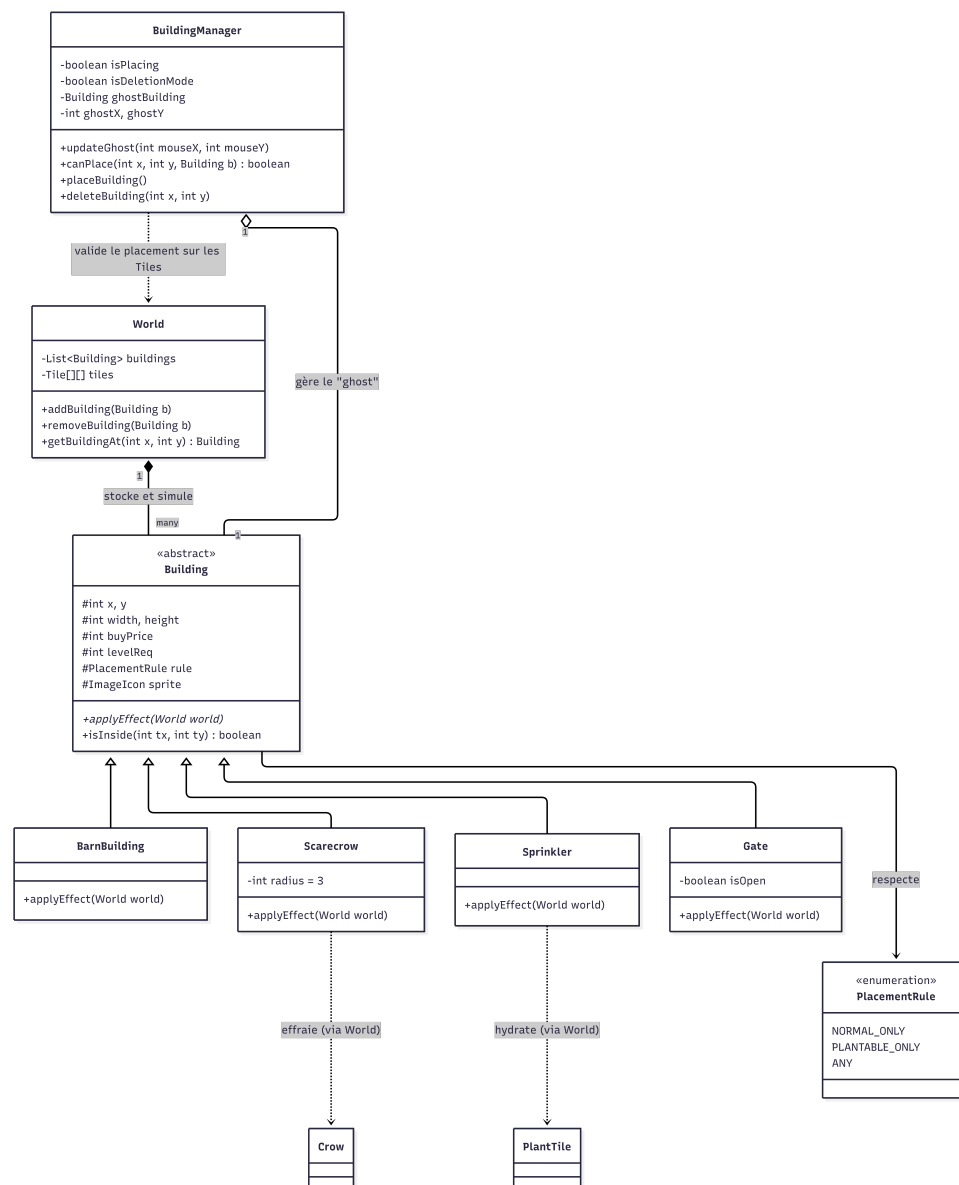


Figure 8 – Diagramme de la fonctionnalité 5.5

5.6 Économie et Progression

La couche d'économie et de progression constitue la boucle de rétention principale de la simulation. Elle a pour objectif de récompenser les actions du joueur, de guider sa découverte des mécaniques via des objectifs clairs, et d'introduire des contraintes de ressources (monnaie, niveaux) pour structurer le défi du jeu.

5.6.1 L'Expérience et le Passage de Niveaux

La progression globale du joueur est quantifiée par une jauge d'expérience déterminant son niveau actuel. Chaque palier franchi représente une étape d'évolution de la ferme.

Structures de données et constantes La gestion de la progression est centralisée au sein de la classe Stats. Les données fondamentales sont trois variables entières : level,

exp et money. La courbe de progression n'est pas linéaire, mais définie par une fonction à paliers codée dans `getExpForNextLevel()`. Les constantes de seuil évoluent : 100 Exp pour le niveau 2, 150 pour les niveaux 3 et 4, 200 pour le 5, puis 250 par défaut pour les suivants. Pour garantir le modèle MVC, la classe Stats utilise des fonctions de rappel typées `IntConsumer` (ex : `levelUpCallback`, `expGainCallback`) qui permettent de notifier la Vue sans importer de composants graphiques dans le Modèle.

Algorithme de progression et de montée de niveau À chaque action récompensée, l'algorithme suivant est déclenché :

- **Acquisition** : L'appel à `addExp(int e)` incrémente la jauge globale. Le callback `expGainCallback` est immédiatement invoqué, déclenchant généralement l'apparition d'un texte flottant (ex : "+15 XP") sur la grille à l'endroit de l'action.
- **Vérification itérative** : La méthode `checkLevelUp()` exécute une boucle `while` (`exp >= getExpForNextLevel()`). Cette boucle soustrait le seuil requis à l'expérience actuelle et incrémente le niveau.
- **Notification** : À chaque incrémentation, le `levelUpCallback` est appelé. Il déclenche la logique d'adaptation du monde (via `syncGardenersForLevel`, ajoutant de nouveaux travailleurs aux niveaux 3 et 7) et invoque l'interface `LevelUpPopup` pour féliciter le joueur de ses nouveaux déblocages.

Conditions limites et robustesse L'utilisation d'une boucle `while` (au lieu d'un simple `if`) garantit que si une récompense massive octroie suffisamment d'expérience pour passer plusieurs niveaux d'un coup, chaque niveau sera validé séquentiellement et les restes d'expérience seront correctement conservés. De plus, il est impossible de perdre de l'expérience : la méthode n'accepte implicitement que des ajouts, prévenant toute régression de niveau.

Utilisation par les autres fonctionnalités La méthode `addExp()` est appelée de manière disséminée dans la logique métier : par la `HarvestAction` (le gain dépendant de la constante `expGain` de chaque `PlantType` récolté) ou par la classe `Quest` lors de la validation d'un objectif. Les callbacks servent quant à eux de déclencheur pour le `FloatingTextManager` de l'UI.



Figure 9 – Screen du level up popup

5.6.2 Le Système de Quêtes et de Chapitres

Pour éviter que le joueur ne se sente perdu, un système de missions dirigées, divisé en chapitres, encadre son évolution.

Structures de données et constantes Le système repose sur une hiérarchie d'objets :

- **Quest** : Représente un objectif unitaire. Elle stocke un objectif (goal), une progression (progress), des récompenses (rewardMoney, rewardExp), et s'appuie sur une énumération Type (PLANT, HARVEST, BUILD, ACTION) pour catégoriser sa nature.
- **QuestLine** : Un chapitre contenant une liste de quêtes et un état de verrouillage (unlocked).
- **Quests** : Le gestionnaire global instancié dans World. Il maintient l'index du chapitre actif (activeLineIndex) et contient des constantes d'actions universelles (ex : ACTION_PLOW_TILE, ACTION_SELL_CAROTTE).

Algorithme de validation et d'avancement :

- **Signalement** : Lorsqu'une action est exécutée dans le jeu (ex : une case est arrosée, un bâtiment est construit), le World invoque une méthode spécifique du moteur de quêtes (ex : onAction(), onBuild()).
- **Filtrage** : Le gestionnaire itère sur toutes les quêtes du chapitre actif (activeLineIndex). Pour chaque quête, un algorithme de correspondance (matchesActionEvent(), matchesBuildEvent()) est évalué.
- **Progression et Récompense** : Si la correspondance est validée, la méthode addPro-

gress(amount) est appelée. L'algorithme utilise $\text{Math.min}(\text{goal}, \text{progress} + \text{amount})$ pour éviter de dépasser l'objectif. Si la quête atteint 100 %, elle bascule en `completed = true`. Le joueur réclame alors sa récompense (déclenchant `grantReward()`).

- **Transition de Chapitre :** Si toutes les quêtes d'une QuestLine sont accomplies, le système déverrouille le chapitre suivant et incrémente `activeLineIndex`.

Conditions limites et robustesse Le système impose une étanchéité absolue entre les quêtes : une action ne doit faire progresser que la quête ciblée. Le typage fort via l'énumération `Type` et la vérification combinée des métadonnées (ex : action de type "Plante" + vérification stricte que `targetPlantType == CAROTTE`) empêche les collisions. De plus, la robustesse économique exige que la méthode `grantReward()` soit protégée par un flag `rewardClaimed` pour éviter qu'une quête terminée ne soit exploitée pour générer de l'argent et de l'expérience à l'infini.

Utilisation par les autres fonctionnalités L'interface graphique `QuestSidePanel` et `QuestMenuController` lisent l'état en temps réel de ces objets pour générer l'arborescence visuelle des chapitres et les barres de progression. Le `SaveController` s'appuie sur la méthode `getProgressSnapshot()` pour visualiser l'état exact des objectifs et pouvoir les restaurer via `restoreProgress()`.

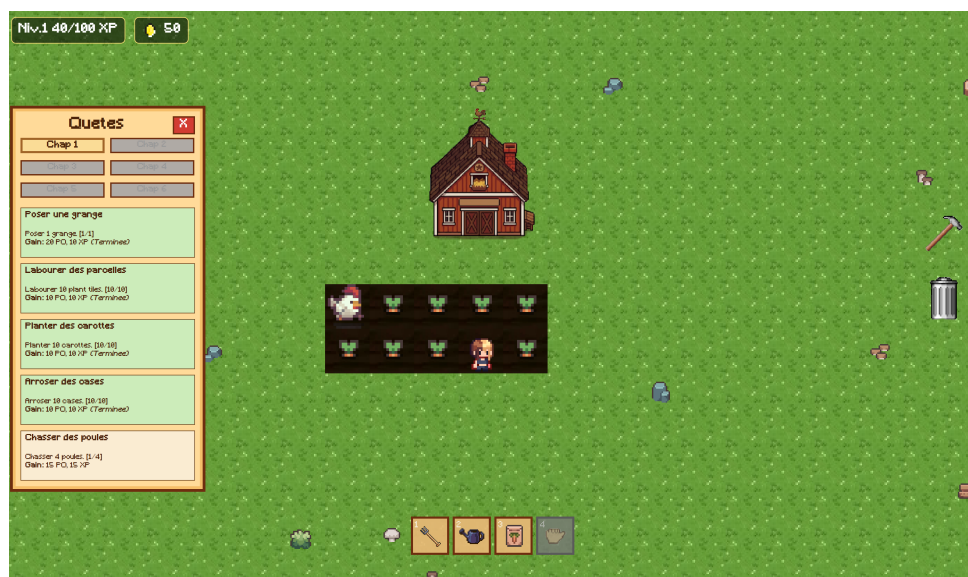


Figure 10 – Screen du panel quête

5.6.3 Le Déblocage des Fonctionnalités

La montée en niveau, au-delà de sa dimension symbolique et de gratification du joueur, est la clé de voûte de la courbe de progression du jeu. Elle est intimement liée à une mécanique d'accès graduel au contenu (graines, infrastructures, et menaces environnementales). Ce système garantit que le joueur assimile les mécaniques de base avant d'être confronté à l'automatisation ou à des attaques de corbeaux.

Structures de Données et Constantes d'Équilibrage L'architecture du déblocage repose sur l'intégration native d'une contrainte d'accès au cœur même des entités du Modèle.

- **Les Plantes (PlantType) :** L'énumération encapsule une variable levelRequirement. L'équilibrage suit une difficulté croissante : la CAROTTE est disponible dès le départ (niveau 0), le CHOUX requiert le niveau 2, la CITROUILLE le niveau 3, et la FRAISE exige le niveau 4.
- **Les Bâtiments (Building) :** La classe abstraite Building intègre également un attribut levelRequirement. Cet attribut est redéfini dans le constructeur de chaque sous-classe pour structurer l'évolution de la ferme : le Puits (Well) et l'Arroseur automatique (Sprinkler) s'acquièrent au niveau 3, les portes (GateFace et GateSide) au niveau 4, l'Épouvantail (Scarecrow) au niveau 5, et les éléments de prestige (Statue) au niveau 6.

Algorithme de Restriction d'Accès et Filtrage UI Contrairement à un système actif, l'algorithme de déblocage est purement passif et consultatif. Il s'appuie sur la Vue et les Contrôleurs pour autoriser ou refuser l'interaction.

- **Filtrage Contextuel :** Lorsque le joueur interagit avec les interfaces marchandes (comme le BuildingSidePanel ou le PopupBarn), l'interface exécute un algorithme de balayage sur le catalogue des éléments (la liste ALL_BUILDINGS ou PlantType.values()). Pour chaque itération, l'algorithme interroge le Modèle via l'instruction world.getStats().getLevel() et compare ce résultat avec la constante levelRequirement de l'objet évalué.
- **Génération Dynamique de l'État Visuel :** Si le niveau du joueur est strictement inférieur au niveau requis (locked = true), l'algorithme de rendu altère la représentation graphique :
 - La carte de l'objet adopte un calque colorimétrique grisé (ex : new Color(170, 160, 150)).
 - L'icône de l'objet est remplacée ou masquée par l'image LOCK_ICON (un cadenas).
 - Le bouton d'action principal subit un .setEnabled(false) physique, bloquant les écouteurs d'événements, tandis que le texte informatif affiche explicitement "Niveau X" pour motiver le joueur.
- **Notification de Niveau :** Lors de l'appel au levelUpCallback, la classe LevelUpPopup invoque sa méthode statique getUnlocksForLevel(int newLevel). Cet algorithme opère une recherche ciblée sur l'ensemble de la base de données de jeu : il filtre uniquement les éléments dont le levelRequirement est strictement égal au nouveau

niveau. Les objets correspondants sont compilés dans une liste d'entités `UnlockEntry` et présentés triomphalement au joueur avec une icône superposée de badge "NEW" (`new.png`), garantissant une interface gratifiante et claire.

Algorithme de Régulation de la Difficulté Environnementale Le système de restriction de niveau ne s'applique pas uniquement à l'économie du joueur, mais orchestre également la difficulté du monde via les Spawners. L'apparition de menaces asymétriques est conditionnée à la progression du joueur. Par exemple, le `CrowSpawner` est implémenté dans son propre Thread. Au sein de sa boucle de génération `run()`, l'algorithme évalue d'abord : `if (world.getStats() != null && world.getStats().getLevel() >= 4)` avant d'invoquer la méthode d'instanciation de l'ennemi.

Logique systémique : Cette barrière technique au niveau 4 correspond exactement au palier d'acquisition des portes hermétiques (Gate) et préfigure l'obtention de l'Épouvantail, assurant que le joueur dispose (ou est sur le point de disposer) de l'arsenal tactique nécessaire pour endiguer la menace volante.

Conditions Limites et Robustesse La robustesse logicielle d'une telle mécanique exige la prise en compte de plusieurs anomalies potentielles :

- **Manipulation d'inventaire et rétrocompatibilité** : Que se passe-t-il si un joueur, à la suite d'un événement externe ou du chargement d'une ancienne sauvegarde, se retrouve en possession d'une graine de fraise (Niveau 4) alors qu'il n'est que Niveau 2 ? L'architecture prévient les conflits en désolidarisant la possession de l'achat. Si l'item existe physiquement dans l'Inventory, la vue de l'inventaire (`PopupBarn`) traitera et affichera la ressource de façon standard (permettant la vente ou la plantation), mais le catalogue d'achats maintiendra la restriction bloquant l'acquisition de nouvelles unités.
- **Synchronisation Concurrente** : Puisque l'expérience est souvent gagnée en arrière-plan via les actions autonomes des entités (`Thread Gardener`), l'appel au `levelUpCallback` peut être déclenché hors du contexte d'affichage. Afin de prévenir tout crash de l'interface (comme une mise à jour d'un composant Swing depuis un thread secondaire), l'algorithme encapsule systématiquement l'appel au popup dans un `SwingUtilities.invokeLater(...)`, garantissant la thread-safety absolue du passage de niveau et de la mise à jour visuelle du catalogue.

Utilisation Transversale par les Autres Fonctionnalités Cette fonctionnalité structure l'ensemble du projet de manière transversale :

- **Le Modèle (Stats)** : Maintient l'état mathématique réel du joueur.
- **La Vue (LevelUpPopup, BuildingSidePanel, PopupBarn)** : Interroge continuellement le niveau en mode passif pour s'assurer que le contenu affiché est légitime.
- **Les Jardiniers (World.syncGardenersForLevel)** : La montée en niveau est

également le déclencheur pour débloquent de nouveaux Jardiniers, rendant l'expansion de la ferme fonctionnellement plus efficace à mesure que la complexité logistique des recettes agricoles augmente.



Figure 11 – Screen du popup barn

5.6.4 Le Système d'Économie :

Le moteur économique simule la circulation des richesses au sein de l'exploitation. La monnaie virtuelle (Pièces d'Or) constitue la ressource névralgique régissant le rythme de développement de la ferme. Ce système a été pensé pour récompenser l'investissement temporel du joueur tout en sécurisant rigoureusement les transactions mathématiques sous-jacentes.

Structures de données et constantes d'équilibrage L'architecture financière repose sur une séparation stricte entre le capital et les actifs matériels.

- **La Monnaie (Stats) :** La trésorerie globale est encapsulée dans la classe Stats via l'attribut entier money.
- **Les Actifs (Barn) :** Les flux transactionnels de marchandises transitent par la classe Barn, qui exploite la structure générique Inventory pour stocker des objets de type ItemSeed (graines) ou ItemPlant (récoltes).
- **Le Modèle Économique (PlantType) :** L'équilibrage financier est statiquement défini dans l'énumération PlantType via l'attribut value. Pour garantir la viabilité de la boucle de gameplay (le "Retour sur Investissement"), les prix sont calculés dynamiquement par une formule mathématique stricte : le prix de vente d'une graine ou d'une plante est fixé à $\text{value} / 2$, tandis que son prix d'achat est majoré à $(\text{value} / 2) + 1$. Cet algorithme force le joueur à accomplir le cycle de croissance

complet (planter, arroser, récolter) pour générer un profit, empêchant toute faille d'achat/revente instantané.

Algorithme de Transaction Les opérations marchandes sont orchestrées par le Barn-Controller, qui agit comme le garant des règles métier lors des interactions avec l'interface de la grange. Le flux transactionnel se divise en deux algorithmes distincts :

- **Algorithme d'Achat (buyItem) :** Le système évalue d'abord la requête en récupérant la quantité souhaitée (qty). Il calcule le coût total de l'opération ($\text{itemPrice} * \text{qty}$). Le contrôleur interroge ensuite la classe Stats : si la condition de solvabilité est respectée ($\text{stats.getMoney()} \geq \text{cost}$), la transaction s'exécute de manière atomique. Le capital est débité ($\text{removeMoney}(\text{cost})$) et le système instancie les nouveaux objets (ItemSeed ou ItemPlant) qui sont immédiatement injectés dans le stockage via $\text{super.addItem}()$.
- **Algorithme de Vente (sellItem) :** L'opération inverse exige une vérification des stocks. Le contrôleur s'assure que le produit cible existe dans la grange et que l'inventaire est suffisant ($\text{item.getQuantity()} \geq \text{qty}$). Si validée, la quantité d'objets est détruite en mémoire ($\text{removeQuantity}(\text{qty})$) et la plus-value calculée ($\text{itemPrice} * \text{qty}$) est créditée sur le portefeuille du joueur ($\text{addMoney}()$).
- **Retour Sensoriel et UX :** La réussite d'une transaction sollicite le SoundManager pour déclencher un retour auditif (clip SELL ou BUY). En cas d'échec (fonds ou stocks insuffisants), l'action est avortée silencieusement en arrière-plan, mais la vue déclenche la méthode statique $\text{GameDialog.showMessageDialog}()$ pour afficher un popup informatif au joueur, explicitant le déficit sans corrompre l'état du jeu.

Conditions limites et robustesse Ce module étant critique, il intègre des sécurités pour prévenir les comportements anormaux ou les tentatives d'exploitation d'exploits :

- **Sanitisation des Quantités :** Lors de la saisie d'une quantité par le joueur, la méthode $\text{parseQuantity}()$ applique un filtre mathématique strict : $\text{Math.max}(\text{parsed}, 1)$. Cette barrière bloque toute injection de valeurs nulles ou négatives. Sans cette protection, un joueur pourrait demander à acheter "-10 graines", ce qui, multiplié par le prix, ajouterait frauduleusement de l'argent à son compte.
- **Sécurité Transactionnelle :** L'algorithme est conçu pour éviter les états instables en cas d'interruption. Lors d'un achat, l'argent n'est déduit qu'une fois la certitude d'ajouter l'objet validé. Cette approche prévient la perte de ressources (le joueur paie mais ne reçoit rien) ou la duplication de biens matériels.

Utilisation par les autres fonctionnalités L'économie est la clef de voûte qui contraint et motive l'ensemble des modules du jeu :

- **Développement (BuildingManager) :** Le gestionnaire de construction intercepte

systématiquement toute action de placement (ex : poser un Arroseur ou un Puits) si le buyPrice du bâtiment excède les fonds disponibles dans Stats.

- **Génération de Richesse (HarvestAction) :** L'action de récolte est l'unique source de création monétaire primaire. En transformant le temps de pousse en un ItemPlant gratuit (stocké directement dans le Barn), elle amorce la chaîne de valeur du joueur.
- **Persistance (SaveManager) :** La valeur exacte de stats.getMoney() est capturée lors du processus de sérialisation. Cette donnée est fondamentale pour geler l'économie locale et la restaurer à l'identique lors de la reprise d'une partie.

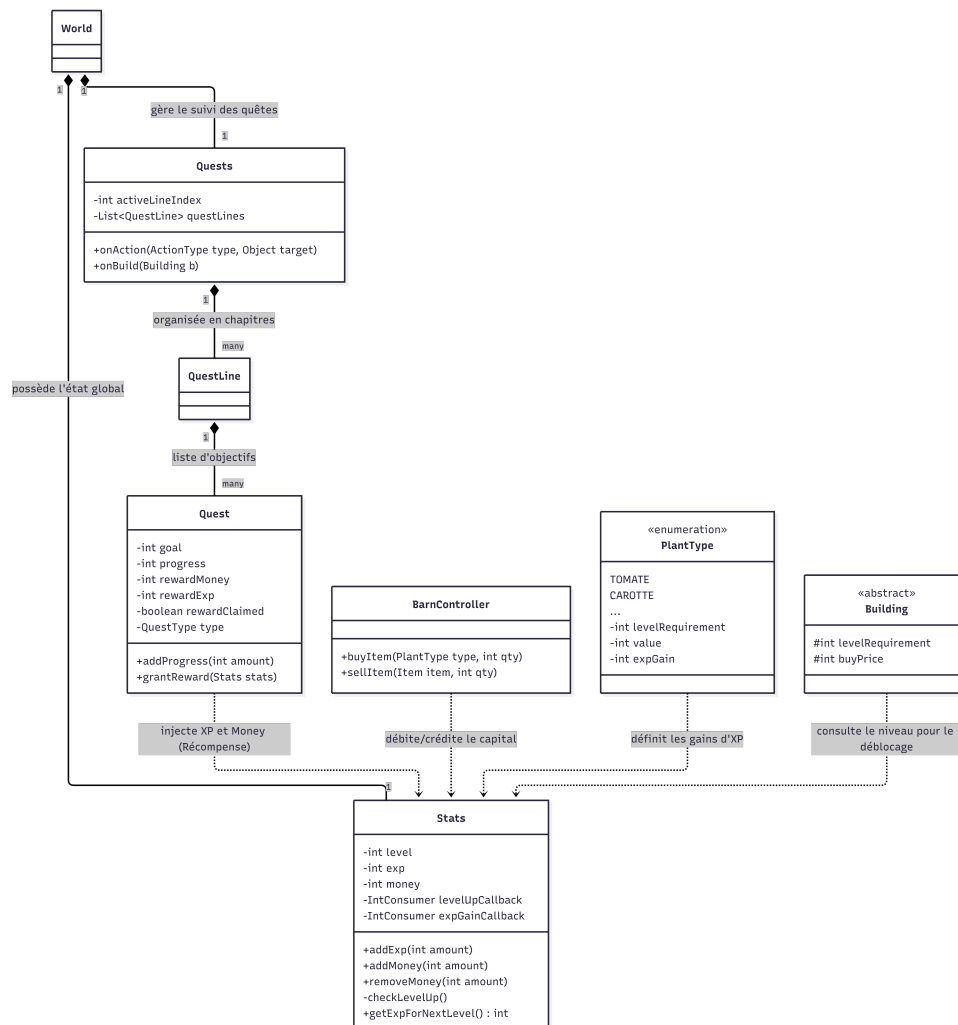


Figure 12 – Diagramme de la fonctionnalité 5.6

5.7 Interface et Contrôles

Cette section décrit l'implémentation de l'interface utilisateur (UI), des menus et des différents états de la simulation. Le système repose sur une machine à états gérée par la classe Display, qui orchestre les transitions entre trois modes principaux :

- **Mode Global** : La caméra est mobile ; le joueur interagit avec la grille, les bâtiments et les entités.
- **Mode Popup** : L'interaction est focalisée sur une fenêtre contextuelle pour consulter des détails ou effectuer des actions (inventaire, magasin).
- **Mode Sélection** : Un mode intermédiaire où le joueur doit désigner une cible spécifique sur la carte, à la demande d'un menu préalable (ex : choisir une case pour l'arrosage).

Il existe aussi le mode construction et le mode suppression, qui sont deux cas particuliers du mode sélection pour poser et respectivement détruire des bâtiments.

5.7.1 Popups Informatifs et Système de Menus

Le système de popups constitue l'ossature de l'interaction riche. Il permet de délivrer des informations ou de gérer des inventaires sans rompre la continuité visuelle de la simulation en arrière-plan.

Architecture et Intégration Visuelle L'affichage repose sur le `JLayeredPane` de la classe `Display`. Cette structure permet de superposer différents calques de profondeur :

- **Le Calque Global (`DEFAULT_LAYER`)** : Il contient la grille, les entités et la barre d'outils. Ces éléments continuent d'être mis à jour visuellement même lorsqu'un menu est ouvert.
- **Le Calque d'Interception (`PopupView` sur le `MODAL_LAYER`)** : Lorsqu'un popup est activé, la classe `PopupView` (un `JPanel` transparent recouvrant l'écran) s'interpose. Elle intercepte tous les événements de la souris (clics, survols), garantissant qu'aucune interaction accidentelle n'ait lieu sur le terrain tant qu'un menu est actif.

Standardisation et Design (La classe `PopupPanel`) Afin de garantir une cohérence esthétique et de limiter la duplication de code, toutes les fenêtres contextuelles héritent de la classe abstraite `PopupPanel`.

- **Contraintes Spatiales** : Elle définit les dimensions fixes (`width`, `height`) et contient la logique de centrage automatique à l'écran.
- **Identité Visuelle** : Elle applique le thème graphique (inspiré de *Stardew Valley*), incluant les bordures (`SDV_BORDER_DARK`) et les fonds texturés (`SDV_BG`).
- **Contrôleur de Fermeture** : Chaque instance génère un bouton de fermeture standardisé, relié au `Display` pour déclencher la méthode `switchToGlobal()`, libérant ainsi les contrôles du terrain.

Algorithme de Transition et Raccourcis Clavier Le passage d'un état à l'autre est

géré par les méthodes `switchToPopup()` et `switchToGlobal()`. L'ergonomie est renforcée par une gestion contextuelle de la touche Échap via `InputMap` et `ActionMap` :

- **Priorité 1** : Si un popup est ouvert, Échap le ferme.
- **Priorité 2** : Si le joueur est en mode construction (`isPlacing`), Échap annule la pose du bâtiment fantôme.
- **Priorité 3** : En mode Global pur, Échap instancie et ouvre le `PauseMenuPopup`.

Extensibilité Grâce à cet héritage, le système propose des implémentations variées répondant à tous les besoins du gameplay :

- **TextPopup** : Utilise un panneau formaté (`TEXT_PADDING`, `TEXT_FONT`) pour les tutoriels et alertes.
- **PopupBarn et PopupInventory** : Génèrent dynamiquement des grilles de boutons (`GridLayout`) pour la gestion des stocks et l'économie.
- **PauseMenuPopup** : Interface de contrôle permettant d'interagir avec le `SoundManager` (musique/bruitages) et de déclencher la sauvegarde.
- **LevelUpPopup** : Combine textes et icônes pour notifier le joueur des nouvelles structures débloquées lors de la progression.

Conditions limites et Robustesse La robustesse du système est assurée par le calque d'interception. Il est techniquement impossible pour un clic de souris d'atteindre une `Tile` si la `PopupView` est active, prévenant les erreurs de logique métier (ex : planter une graine alors que l'on parcourt le menu de pause). De plus, le centrage des popups est recalculé dynamiquement lors du redimensionnement de la fenêtre de jeu pour éviter que l'UI ne devienne inaccessible ou partiellement hors-champ.

Utilisation par les autres fonctionnalités Ce système est le réceptacle de sortie de nombreuses logiques : le `SaveManager` l'utilise pour confirmer le succès d'une écriture, le système de progression pour célébrer un gain de niveau, et les `ActionBuilders` (`Plantation`, `Récolte`) pour permettre au joueur de choisir ses ressources avant de lancer une tâche.

5.7.2 Barre d'Action Rapide (Hotbar) et Équipements

Afin de fluidifier l'expérience utilisateur et de minimiser les allers-retours dans des menus complexes, ainsi que pour minimiser les clics, le jeu intègre une barre d'action rapide. Affichée en surimpression au bas de l'écran, elle permet au joueur de visualiser instantanément ses outils disponibles et de basculer rapidement d'une action à l'autre. La hotbar contient donc les 4 actions que le joueur peut donner aux jardiniers.

Architecture et Intégration Visuelle (La Vue) Sur le plan architectural, la Hotbar est gérée directement par la classe `Global`, qui est le conteneur principal du rendu graphique.

La méthode dédiée `drawHotbar(Graphics g)` est appelée à la toute fin de la boucle de rendu `paintComponent()`. Ce placement stratégique garantit que la barre d'action est toujours dessinée par-dessus le terrain, les entités et les autres éléments du monde, assurant sa visibilité en toutes circonstances. De plus, un booléen `hotbarVisible` permet de masquer dynamiquement l'interface (par exemple, lors de l'ouverture d'un autre menu de type `PopupView` ou lors de cinématiques) afin de ne pas surcharger l'écran.

Structure, Design et Ergonomie Le design de la Hotbar a été pensé pour offrir un rendu esthétique et intuitif, en s'inspirant des standards du jeu de gestion agricole (comme *Stardew Valley*) croisés avec la lisibilité d'interfaces classiques (comme *Minecraft*).

- **Les Emplacements (Slots) :** La barre est constituée de 4 emplacements de taille fixe (52x52 pixels), espacés uniformément et centrés dynamiquement en bas de la fenêtre grâce aux calculs $(getWidth() - totalWidth) / 2$.
- **Transparence et Couleurs :** Pour ne pas occulter totalement le terrain sous-jacent, le fond des cases utilise une couleur RGBA semi-transparente (Sable).
- **Feedback Visuel (Highlight) :** L'interface indique clairement l'outil actuellement équipé. Les cases inactives possèdent une bordure fine et sombre (marron). La case sélectionnée, en revanche, voit sa bordure s'épaissir (passage d'un `BasicStroke` de 3 à 5) et basculer en blanc pur, guidant immédiatement l'œil du joueur.
- **Repères d'interaction :** Chaque case affiche un raccourci numérique (1 à 4) dans son coin supérieur gauche, utilisant la police personnalisée du jeu (`MINECRAFT_FONT`), indiquant explicitement au joueur quelles touches utiliser pour interagir.

Mécanique de Sélection et Contrôle (Le Contrôleur) La Hotbar est le reflet direct de l'état du joueur (le premier *Gardener* de la liste). La méthode de rendu récupère dynamiquement l'index de l'outil actif via `player.getSelectedHotbarIndex()`. Les contrôleurs du jeu (via des écouteurs de type `KeyListener` ou `MouseWheelListener`) captent les entrées du joueur (pressions sur les touches 1, 2, 3, 4 ou défilement de la molette) pour mettre à jour cet index dans le Modèle. À chaque modification, une requête de rafraîchissement (`repaint()`) est envoyée à la vue, garantissant une mise à jour visuelle instantanée du surlignage.

Détail des Fonctionnalités et Outils Intégrés Chaque emplacement de la Hotbar correspond à une fonctionnalité majeure du cycle agricole, chargée en mémoire sous forme d'image (`Image`) lors de l'initialisation de `Global` :

- **Emplacement 1 : La Houe (`houeImg`)** Cet outil est lié au contrôleur `PlowActionSelector`. Sélectionner la houe permet au joueur d'interagir avec les cases de terre vierge (`Tile : isPlowable`) pour les transformer en parcelles cultivables (`PlantTile`).
- **Emplacement 2 : L'Arrosoir (`arrosoirImg`)** Lié au contrôleur `WaterActionSelec-`

tor. L'arrosoir est l'outil de survie des plantes. Il permet de cibler n'importe quelle parcelle contenant une plante vivante (de la graine à la maturation) pour restaurer son niveau d'eau et relancer sa croissance.

- **Emplacement 3 : Le Sac de Graines / Plantation (`planterImg`)** Cet emplacement déclenche le processus de plantation (`PlantActionSelector`). Sélectionner cet outil ouvre le menu de l'inventaire (`PopupInventory`) pour permettre au joueur de choisir quelle graine (Carotte, Salade, Tomate...) il souhaite extraire de son stock pour l'enfouir dans le sol. Il sélectionne ensuite une parcelle sur laquelle planter.
- **Emplacement 4 : La Récolte (`recolterImg`)** Lié au contrôleur `HarvestActionSelector`. Cet "outil" représente l'action manuelle du jardinier. Il possède une double fonctionnalité selon l'état de la cible :
 - Si la plante est à l'état MATURE, il procède à son arrachage et ajoute le produit dans l'inventaire économique de la grange.
 - Si la plante est à l'état MORT (suite à un manque d'eau ou une attaque d'ennemi), il nettoie la parcelle pour libérer la terre, agissant comme un outil de réhabilitation.

5.7.3 Interaction Spatiale et Retours Visuels

Alors que les popups et la hotbar gèrent les actions immédiates, le système d'interface s'étend à des modes d'interaction complexes pour la gestion du terrain et le suivi de la progression. Ces mécaniques reposent sur une modification temporaire de la machine à états du Display et sur des surcouches graphiques informatives.

5.7.3.1. Le Mode Sélection : Logique et Interaction Contextuelle

Le mode Sélection est un état de contrôle hybride activé par les contrôleurs d'action (ex : planter, arroser). Il permet de désigner précisément les zones d'intervention des jardiniers.

- **Contrôles Avancés** : L'interaction dépasse le simple clic unitaire. L'implémentation gère le Shift+Clic, permettant de sélectionner automatiquement une parcelle entière (groupe de tuiles adjacentes de même type), optimisant ainsi la gestion des grandes exploitations.
- **Validation des Saisies** : Le flux est sécurisé par des raccourcis clavier contextuels mappés dans le `SelectionController` : la touche Entrée confirme la transaction et génère les actions dans la file d'attente, tandis que Échap annule l'opération et ramène le joueur en mode Global.

5.7.3.2. Feedback Visuel Dynamique et Surlignage

Pour assurer une lisibilité parfaite de l'état de la ferme, le moteur de rendu utilise des codes couleurs et des indicateurs flottants :

- **Surlignage de sélection** : En mode sélection, les tuiles ciblées sont entourées d'un liseré bleu. Une fois l'action validée par le joueur, le surlignage bascule au jaune pour indiquer qu'une intervention est planifiée.
- **Intensité cumulative** : Le système de rendu calcule l'épaisseur ou l'opacité du jaune en fonction du nombre de jardiniers assignés à une même case. Plus l'action est priorisée par plusieurs entités, plus le retour visuel est intense, permettant au joueur d'identifier les goulets d'étranglement logistiques.
- **Indicateurs de gain (Floating Feedback)** : Lors des récoltes, ventes ou achèvements de quêtes, la classe FloatingTextManager génère de petits indicateurs visuels au-dessus des entités concernées. Ces textes affichent les quantités d'expérience (XP) et de monnaie (PO) gagnées avant de s'estomper, offrant une gratification immédiate.

5.7.3.3. Panneaux Latéraux : Quêtes et Construction

Pendant le mode Global, deux panneaux rétractables permettent la gestion stratégique de la ferme :

- **Panneau de Quêtes (QuestSidePanel)** : Situé à gauche de l'écran, il s'active via une icône dédiée. Il organise les missions par chapitres et affiche les descriptifs ainsi que les barres de progression en temps réel.
- **Panneau de Construction (BuildingSidePanel)** : Situé à droite, il répertorie les structures classées par catégories (Économie, Défense, Décoration). Chaque entrée affiche une icône, une description et le prix. Si le niveau requis n'est pas atteint, un cadenas remplace le bouton de pose, protégeant ainsi la progression.

5.7.3.4. Modes Construction et Suppression

Ces deux modes sont des extensions logiques du mode Sélection, utilisant des algorithmes de validation spécifiques :

- **Mode Construction** : Il introduit le système de preview. Une version translucide du bâtiment suit le curseur, vérifiant à chaque déplacement la validité du terrain (canPlace). Le passage au rouge du fantôme indique une collision ou un sol incompatible.
- **Mode Suppression** : Déclenché par l'icône poubelle, ce mode permet de sélectionner des bâtiments ou obstacles. Pour éviter toute destruction accidentelle, la validation finale n'est effectuée qu'après la confirmation via un TextPopup de validation, garantissant la sécurité des infrastructures coûteuses.

5.7.3.5. Système de Tutoriel et Accompagnement

Le jeu intègre une séquence de tutoriel basée sur le polymorphisme des TextPopup. Ces fenêtres informatives se déclenchent lors de la première session, isolant les notions

principales (labourage, arrosage, vente). Elles invitent le joueur à effectuer des actions spécifiques dans un environnement contrôlé avant de lui laisser la pleine autonomie sur l'exploitation.

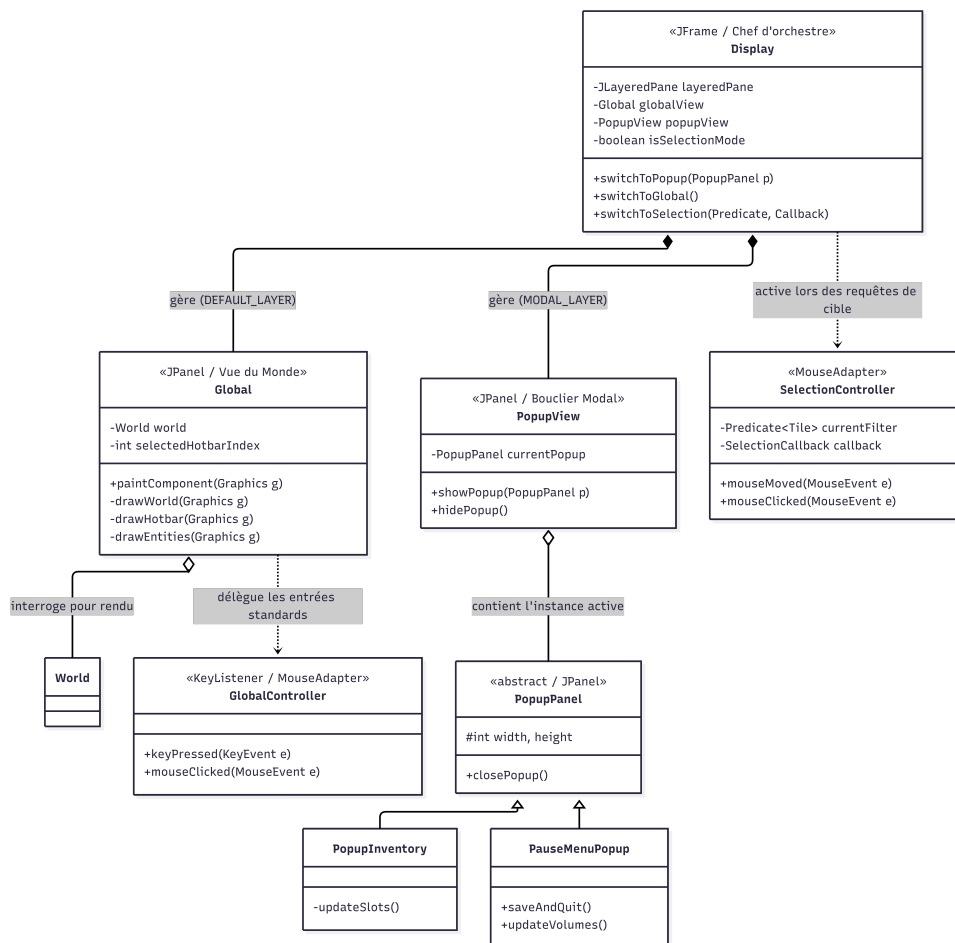


Figure 13 – Diagramme de la fonctionnalité 5.7

5.8 Menu et Sauvegarde

Cette section détaille l'implémentation de la fonctionnalité F8, consacrée à la navigation globale dans l'application et à la persistance des données. Au démarrage du jeu, le joueur interagit avec un menu afin de soit lancer une nouvelle partie soit charger une sauvegarde existante. Cette mécanique garantit la pérennité de l'expérience joueur, tout en assurant une gestion stricte du cycle de vie des Threads.

5.8.1 Écran d'Accueil et Navigation

L'écran d'accueil est l'interface initiale permettant de naviguer entre la création d'une nouvelle exploitation, la reprise d'une partie existante ou la configuration des options.

Réalisation et Structures de Données

Le menu principal repose sur une séparation claire entre le conteneur principal (Main) et la vue du menu (HomeScreenPanel).

- **Main (Classe d'entrée)** : Agit comme le chef d'orchestre global de l'application (le Routeur). Il possède la référence unique à la JFrame et orchestre le passage exclusif entre l'écran d'accueil et le moteur de jeu (Display).
- **HomeScreenPanel** : Hérite de JPanel. Cette vue gère l'affichage du fond d'écran et du bouton principal interactif ("Jouer"). Elle délègue la logique métier (le lancement concret d'une partie) via une interface fonctionnelle (Runnable), injectée par le Main lors de sa construction.
- **SaveSelectionPopup** : Fenêtre de dialogue modale (JDialog) qui liste les fichiers de sauvegarde détectés sur le disque et permet au joueur de saisir le nom d'une nouvelle exploitation. La création d'une partie s'accompagne d'une initialisation avec les paramètres par défaut (ressources de départ, terrain vierge).

Gestion des Transitions et Cycle de Vie

Afin d'éviter les fuites de mémoire et la surcharge du processeur, la transition du Jeu vers le Menu implique un nettoyage complet des instances actives :

- **Désallocation** : Lors du retour au menu, le Timer gérant la boucle de rendu (Rendering) est annulé (`currentTimer.cancel()`) et la référence au Display est détruite.
- **Callback de retour** : Le Display possède une méthode `setReturnToMenuCallback(Runnable callback)` qui permet d'injecter la commande de retour à l'écran d'accueil. Ce couplage faible permet au bouton "Quitter" du menu de pause de déclencher le retour au menu sans que le Display n'ait besoin de connaître la classe Main.
- **Transition Musicale** : Le SoundManager orchestre un fondu audio (fade-out / fade-in) de 1000 millisecondes pour basculer en douceur entre la piste ambiante du menu et la musique du jeu.

5.8.2 Architecture de la Sauvegarde

La persistance des données dans une simulation systémique comme celle-ci représente un défi architectural majeur. Chaque sauvegarde doit capturer l'état instantané d'un environnement hétérogène (grille, entités biologiques, infrastructures, économie) pour permettre une reconstruction fidèle lors de la reprise du jeu.

Problématique de la Sérialisation et Structures de Données En Java, l'approche naïve consistant à rendre la classe World directement Serializable est impraticable. Le World contient des références vers des composants graphiques (comme BufferedImage ou ImageIcon), des processus actifs (instances de Thread pour les entités) et des références circulaires complexes. Ces éléments ne sont pas sérialisables et provoquent des exceptions `NotSerializableException`.

Pour contourner cela, l'architecture extrait les données pures dans des structures dédiées :

- **WorldSaveData** : La racine du DTO. Elle implémente Serializable et fait office de conteneur pour les primitives de progression (level, exp, money) et les listes de sous-DTO.
- **PlantTileSaveData** : Une structure légère représentant l'état d'une parcelle cultivée. Elle ne stocke pas l'objet Plant mais ses attributs primitifs : coordonnées (x, y), âge (age), niveau d'hydratation (currentWaterLevel) et l'identifiant de l'espèce (plantTypeName).
- **BuildingSaveData** : Pour les infrastructures, le système stocke les coordonnées et, de manière cruciale, le nom de la classe sous forme de String (className).
- **Constantes du modèle** : Le chemin de stockage par défaut (souvent un répertoire saves/) et le suffixe de fichier .ser utilisé pour identifier les archives binaires.

Algorithme de Sauvegarde Le cycle de vie d'une sauvegarde se décompose en deux algorithmes distincts gérés par le SaveManager :

- **Algorithme d'Extraction (Écriture)** :
 - **Initialisation** : Création d'une instance de WorldSaveData à partir de l'état actuel du World.
 - **Mapping des entités** : Le système parcourt la matrice des tuiles. Pour chaque PlantTile contenant une plante, il instancie un PlantTileSaveData. Il itère ensuite sur la liste des bâtiments (World.getBuildings()) pour générer les BuildingSaveData.
 - **Sérialisation** : L'objet WorldSaveData est passé à un ObjectOutputStream lié à un FileOutputStream. Java transforme alors ce graphe de données primitives en un flux binaire stocké sur le disque.
- **Algorithme de Reconstruction (Lecture et Réflexion)** :
 - **Désérialisation** : Lecture du fichier via ObjectInputStream pour recréer l'objet WorldSaveData en mémoire.
 - **Réinstanciation par Réflexion** : C'est l'étape la plus technique. Pour chaque BuildingSaveData, le système utilise l'API de Réflexion Java (Class.forName(className)) pour retrouver la classe correspondante et invoquer son constructeur. Cela permet de reconstruire dynamiquement n'importe quel type de bâtiment (Grange, Puits, Épouvantail) sans avoir de structure switch/case rigide.
 - **Restauration Biologique** : Les plantes sont recrées à partir de leur PlantType et leurs compteurs d'âge et d'eau sont réinjectés, remplaçant la plante exactement dans son état de croissance initial.

- **Rechargement des Assets** : Puisque les images étaient exclues du DTO, une méthode de post-traitement lie à nouveau chaque entité à ses ressources graphiques via le ResourceManager.

Conditions limites et Robustesse Plusieurs gardes-fous sont indispensables pour garantir l'intégrité des sauvegardes :

- **Gestion des versions** : Pour éviter les crashes si la structure des classes change entre deux versions du jeu, chaque DTO possède un identifiant de version.
- **Inexistence du fichier** : Le `SaveManager.savExists()` vérifie la présence du fichier avant toute tentative de lecture pour éviter les `FileNotFoundException`.
- **Corruption des données** : L'ensemble du processus est encapsulé dans des blocs try-catch gérant les `IOException` et `ClassNotFoundException`, permettant au jeu de retourner au menu principal avec un message d'erreur plutôt que de planter brutalement.
- **Monde vide** : L'algorithme doit gérer les listes vides (aucune plante, aucun bâtiment) sans générer de pointeurs nuls lors de la reconstruction.

Utilisation par les autres fonctionnalités L'architecture DTO est le point de passage obligé pour la continuité de l'expérience :

- **Main** : Lors du lancement, le Main utilise le `SaveManager` pour décider s'il doit initialiser un nouveau World ou charger une `WorldSaveData` existante.
- **PauseMenuPopup** : Le bouton "Sauvegarder" déclenche l'algorithme d'écriture de manière asynchrone pour ne pas geler l'interface.
- **Stats** : L'économie et le niveau du joueur dépendent entièrement de la restauration fidèle des primitives stockées dans le `WorldSaveData`.
- **BuildingManager** : Lors de la reconstruction, le manager doit s'assurer que les propriétés physiques des tuiles (comme `isWalkable = false` sous un bâtiment) sont recalculées après la réinstanciation par réflexion.

5.8.3 Processus de Sérialisation et Flux d'Entrée/Sortie (SaveManager)

Le `SaveManager` est la pièce maîtresse logistique de la persistance des données. Tandis que le patron DTO établit le format des données à sauvegarder, le processus de sérialisation définit la méthode par laquelle ces données sont transférées entre la mémoire vive (RAM) et le support de stockage persistant (disque dur). Ce module est responsable de l'orchestration des opérations d'Entrée/Sortie (I/O) complexes, tout en assurant l'intégrité de la matrice du monde.

Structures de Données et Ressources I/O

Le processus s'appuie nativement sur l'API java.io pour manipuler les flux binaires.

- **Les Flux de Données** : Le système utilise la combinaison d'un `FileOutputStream` (pour l'accès matériel au fichier) encapsulé dans un `ObjectOutputStream` (pour la conversion du graphe d'objets Java en flux d'octets). L'opération inverse utilise logiquement `FileInputStream` et `ObjectInputStream`.
- **Le Conteneur Racine** : Le `WorldSaveData` (DTO racine) sert de réceptacle unique. Transmettre un seul objet racine au flux binaire garantit une écriture atomique et simplifie la reconstruction.
- **Constantes Locales** : Le système de sauvegarde s'appuie sur des chemins relatifs normalisés (ex : un répertoire `saves/` situé à la racine du projet) et identifie les fichiers valides via une extension spécifique (ex : `.sav` ou `.ser`).

Algorithme d'Écriture

L'algorithme de sauvegarde agit comme une capture instantanée (snapshot) du modèle. Invoqué par le joueur, le `SaveManager` exécute la séquence suivante :

- **Initialisation du Conteneur** : Un objet `WorldSaveData` vierge est instancié.
- **Balayage Topologique (Grille)** : Le gestionnaire exécute une double boucle itérative sur la matrice bidimensionnelle du monde (`world.getTiles()`). Pour chaque case évaluée, si elle est de type `PlantTile` (qu'elle contienne une plante vivante, morte, ou qu'elle soit simplement labourée), l'algorithme extrait ses primitives, génère un `PlantTileSaveData` et l'ajoute à la liste du conteneur.
- **Balayage Architectural (Bâtiments)** : Le système parcourt la liste dynamique des infrastructures (`world.getBuildings()`). Il extrait la position spatiale et le nom qualifié de la classe (via `getClass().getName()`) pour générer les `BuildingSaveData`.
- **Extraction Économique** : Les statistiques globales (le niveau actuel du joueur, l'expérience accumulée et la trésorerie gérée par la Barn) sont copiées directement dans les attributs de la racine DTO.
- **Écriture et Libération** : L'objet `WorldSaveData` finalisé est poussé dans le flux binaire `ObjectOutputStream`. L'algorithme force ensuite la vidange du tampon (`flush()`) et la fermeture du flux (`close()`) pour libérer le fichier au niveau du système d'exploitation.

Algorithme de Lecture et Reconstruction

L'algorithme de chargement est asymétrique à celui de l'écriture : il est nettement plus complexe car il doit opérer une rétro-ingénierie pour repeupler un environnement vierge.

- **Désérialisation** : L' `ObjectInputStream` decode le fichier binaire pour instancier l'objet `WorldSaveData` en mémoire.

- **Restauration Biologique (Tuiles et Plantes) :** L'algorithme itère sur la liste des PlantTileSaveData. Pour chaque entrée, il écrase la tuile standard de la grille aux coordonnées (x, y) par une nouvelle instance de PlantTile. Si la sauvegarde indique la présence d'une plante, le système réinstancie l'objet Plant avec le PlantType correspondant. Étape cruciale : l'algorithme réinjecte manuellement les compteurs temporels exacts (age, currentWaterLevel, ticksWithoutWater), resynchronisant ainsi parfaitement l'automate à états finis de la plante.
- **Restauration Architecturale (Réflexion Java) :** C'est ici qu'intervient la Reflection API. En lisant la chaîne de caractères className depuis le BuildingSaveData, le système retrouve la classe dans le ClassLoader et invoque dynamiquement son constructeur.
- **Synchronisation de la Physique (Pathfinding) :** Une fois le bâtiment réinstancié, il est ajouté au World. Cette action n'est pas qu'un ajout logique : elle déclenche instantanément la mise à jour de la grille de navigation (algorithme A*), redéfinissant la propriété isWalkable = false sur les cases couvertes par le bâtiment reconstruit, garantissant que les entités ne traverseront pas les murs chargés.

Conditions Limites et Robustesse

La manipulation de fichiers locaux expose l'application à de nombreuses anomalies matérielles ou logicielles. La robustesse est assurée par un contrôle strict des erreurs (gestion des exceptions) :

- **Inexistence du Répertoire Cible :** Avant toute ouverture de FileOutputStream, l'algorithme d'écriture vérifie la présence du dossier saves/. S'il n'existe pas, il est généré à la volée via File.mkdirs(), prévenant ainsi une erreur silencieuse ou un plantage d'accès (Access Denied).
- **Corruption de Fichier et Rétrocompatibilité :** Lors de la lecture, si le fichier a été altéré ou s'il appartient à une version antérieure incompatible du jeu, l'API lèvera une InvalidClassException ou ClassNotFoundException. Ces exceptions sont capturées dans un bloc catch. Le système purge alors les données partiellement chargées et notifie la Vue (ex : via un popup d'erreur "Fichier corrompu") pour renvoyer le joueur au menu principal de manière sécurisée.
- **Sécurité des Ressources :** L'utilisation de blocs try-with-resources ou finally garantit que les flux (InputStream/OutputStream) sont obligatoirement fermés, même si une exception interrompt l'algorithme au milieu du processus. Cela empêche l'OS de verrouiller le fichier de sauvegarde indéfiniment.

Utilisation par les Autres Fonctionnalités

Le SaveManager est une "boîte noire" transactionnelle utilisée par les couches supérieures du jeu :

- **Le Routeur Global (Main)** : Utilise l'algorithme de Lecture lors du basculement du HomeScreenPanel vers le Display si le joueur a sélectionné un fichier existant dans le SaveSelectionPopup.
- **L'Interface de Contrôle (PauseMenuPopup)** : Utilise l'algorithme d'Écriture. Lorsque le joueur clique sur "Sauvegarder et Quitter", ce menu déclenche le processus avant d'exécuter la séquence de retour au menu, assurant une expérience utilisateur fluide et sécurisée.
- **L'Arbre des Quêtes (QuestManager)** : (Si implémenté dans le DTO), le processus permet de figer l'état exact du chapitre en cours, garantissant que le joueur ne perde pas sa progression dans les missions scénarisées.

5.8.4 Mécanisme de Pause et Suspension d'Exécution

La persistance des données dans un environnement multithreadé exige une synchronisation temporelle absolue. Il est impératif que l'univers simulé soit figé pour éviter qu'une modification asynchrone (ex : une plante qui périt de déshydratation ou un nuisible qui détruit une culture) ne survienne pendant le processus d'écriture sur le disque (I/O Operations). Ce mécanisme de pause garantit l'intégrité de la capture d'état et assure un arrêt propre des ressources matérielles allouées par la Machine Virtuelle Java.

Le Menu de Pause (PauseMenuPopup) et l'Interface de Contrôle

Le menu de pause agit comme le centre de commande systémique du joueur.

- **Interception Logique** : Invoqué par la pression de la touche "Échap" (interceptée par les Key Bindings du Display), ce menu s'affiche sur le calque d'interception (MODAL_LAYER). Son apparition interrompt immédiatement les interactions physiques avec la carte, suspendant ainsi les entrées utilisateur.
- **Gestion Paramétrique** : Le menu intègre des contrôleurs dynamiques reliés au SoundManager. Il permet l'ajustement en temps réel des volumes (musique d'ambiance et bruitages) avec mémorisation de l'état pour personnaliser l'expérience sensorielle.
- **Sécurisation des Intentions** : Le bouton critique "Sauvegarder & Quitter" ne déclenche pas l'arrêt immédiat. Afin de prévenir les erreurs de manipulation, il invoque la classe utilitaire GameDialog pour générer une fenêtre modale de confirmation. Ce motif de conception garantit que la destruction de la session en cours est un acte intentionnel.

Algorithme d'Arrêt Sécurisé

Dès lors que le joueur valide son intention de quitter l'exploitation, le système déclenche un algorithme d'arrêt en cascade, conçu pour éviter toute corruption mémoire ou création de fuite de mémoire :

- **Enregistrement** : Avant de détruire les objets, l'état instantané et statique du jeu est capturé et confié au SaveManager (voir 5.8.3). La grille et les inventaires sont sérialisés en toute sécurité puisque plus aucune entité n'est autorisée à les modifier.
- **Arrêt du Moteur (world.stopWorld())** : La méthode stopWorld() est la clef de voûte de la désallocation. Elle parcourt systématiquement l'ensemble des collections gérant les entités dynamiques (Jardiniers, Poules, Corbeaux, et les Spawners générateurs).
- **Interruption Thread-Safe** : Pour chaque entité Runnable, la méthode stop() est appelée. Cet appel modifie le drapeau de contrôle (isRunning = false) et, de manière cruciale, émet une interruption matérielle via thread.interrupt(). Cette directive est indispensable : elle force les threads qui seraient bloqués dans un état d'attente (Thread.sleep(), utilisé pour simuler le temps de trajet ou l'attente d'une action) à se réveiller immédiatement en levant une InterruptedException. Le thread peut alors sortir proprement de sa boucle run() et mourir naturellement.
- **Désallocation** : Les processus secondaires, tels que le thread du CameraController responsable du défilement continu sur les bordures de l'écran, sont également arrêtés de manière explicite, libérant l'intégralité des cycles CPU alloués au moteur de jeu.

Conditions Limites, Concurrency et Robustesse

Le système de suspension et de persistance intègre des garde-fous stricts pour pallier les anomalies liées au multithreading et aux manipulations de fichiers externes :

- **Prévention des Exceptions Concurrentes** : Figer logiquement le jeu avant la sauvegarde est la seule méthode pour éviter la redoutée ConcurrentModificationException. Si le SaveManager itérait sur la liste des plantes pendant qu'une poule (sur un autre thread) supprimait une plante mangée de cette même liste, l'application crasherait instantanément. Le mécanisme de Pause agit comme un verrou global implicite.
- **Vérification d'Intégrité Préventive** : Lors de la tentative de reprise d'une partie (depuis l'écran d'accueil), l'interface utilise la méthode statique SaveManager.savExists() pour vérifier l'intégrité de l'accès matériel avant d'autoriser la transition. Si le fichier est introuvable (supprimé manuellement par l'utilisateur via son OS), le système dévie le flux et génère automatiquement un World vierge sans lever de FileNotFoundException.
- **Tolérance aux Pannes** : Le processus de lecture de fichier est le plus vulnérable aux corruptions. En cas de fichier illisible (bytes corrompus) ou d'incompatibilité majeure de version (modification de la structure des classes entre deux mises à jour), l'exécution entre dans les blocs catch dédiés aux IOException et ClassNotFoundException. En cas d'erreur de chargement, le jeu revient au menu sans planter.

Utilisation par les Autres Fonctionnalités

Le mécanisme d'arrêt sécurisé est le point de convergence final de la boucle de gameplay :

- **Le Routeur Global (Main) :** Il attend passivement que le callback de fin de session soit invoqué par ce mécanisme pour nettoyer la JFrame et réafficher le HomeScreenPanel.
- **Gestion de l'Énergie :** Sur des machines portables, ce nettoyage strict des processus garantit que l'application ne consomme aucune ressource processeur en arrière-plan une fois retournée au Menu Principal. Cela évite de laisser des threads actif après le retour au menu.

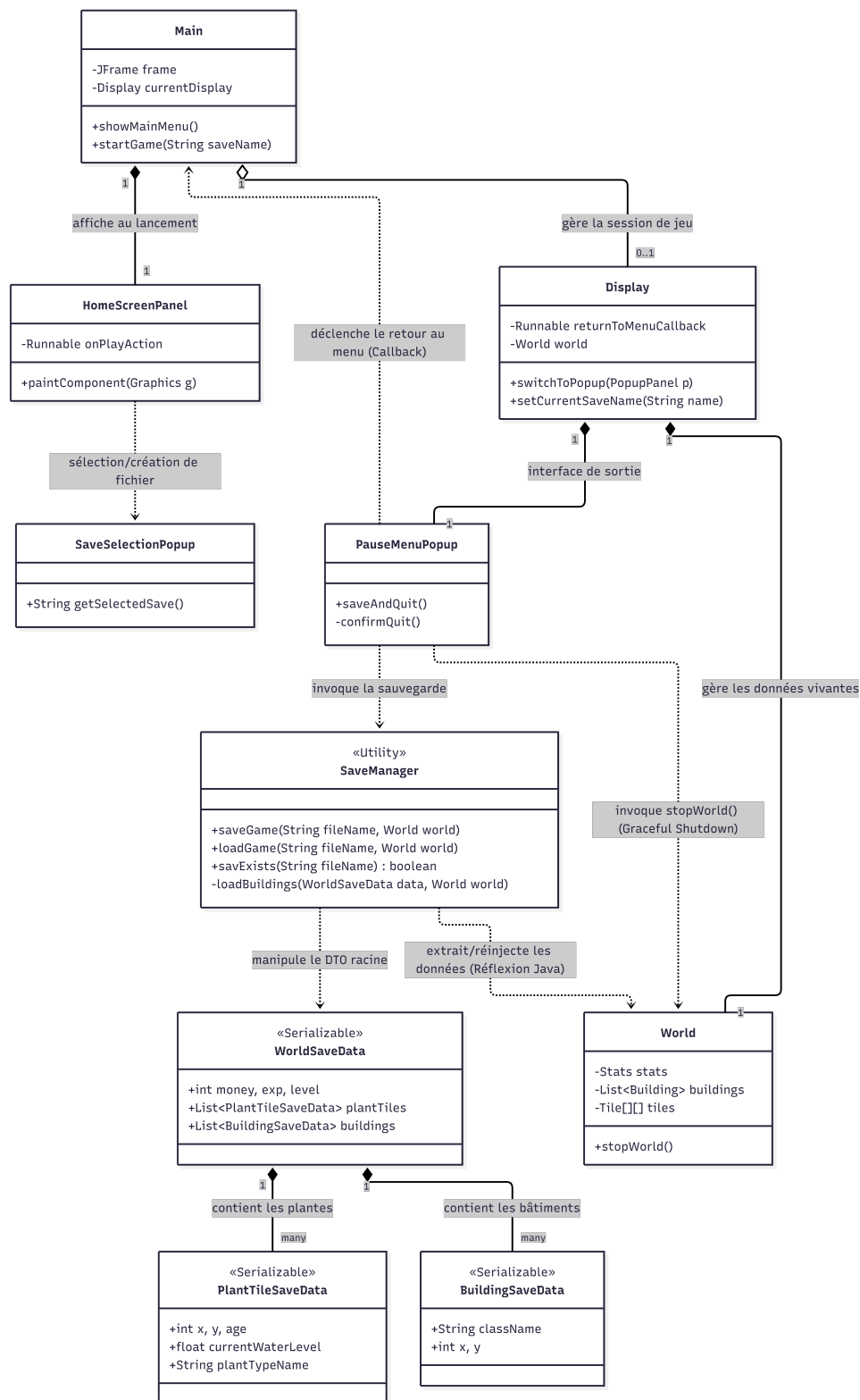


Figure 14 – Diagramme de la fonctionnalité 5.8

6 Résultat : Saclay Valley

Le produit final obtenu est une application Java robuste, offrant une simulation de gestion agricole complète et immersive. Baptisé "Saclay Valley", le logiciel parvient à fusionner les mécaniques classiques des jeux de gestion (inspiré de Stardew Valley).

6.1 Aperçu Global et Expérience Utilisateur

Visuellement, le jeu propose une esthétique soignée en pixel art, structurée autour d'une grille de tuiles réactive. Le joueur évolue sur une carte vaste, naviguant avec fluidité grâce à un système de caméra par défilement bord-écran. L'interface utilisateur se veut minimaliste mais informative : la barre d'action rapide au bas de l'écran permet un contrôle instantané des outils, tandis que les popups contextuels (Grange, Inventaire, Quêtes) s'intègrent de manière cohérente à l'univers visuel sans jamais rompre l'immersion.



Figure 15 – Aperçu global d'une partie en fin de jeu

6.2 Comportement Systémique et Fluidité

Techniquement, le résultat se distingue par sa fluidité d'exécution. Malgré la présence de multiples threads gérant les entités en arrière-plan (jardiniers, poules, corbeaux), le moteur de rendu maintient un taux de rafraîchissement stable de 60 FPS. L'intelligence artificielle des jardiniers est l'un des points forts du produit : les jardiniers calculent des chemins optimaux via l'algorithme A* pour contourner les bâtiments et les barrières, offrant un spectacle de "fourmilière" particulièrement plaisant à regarder.

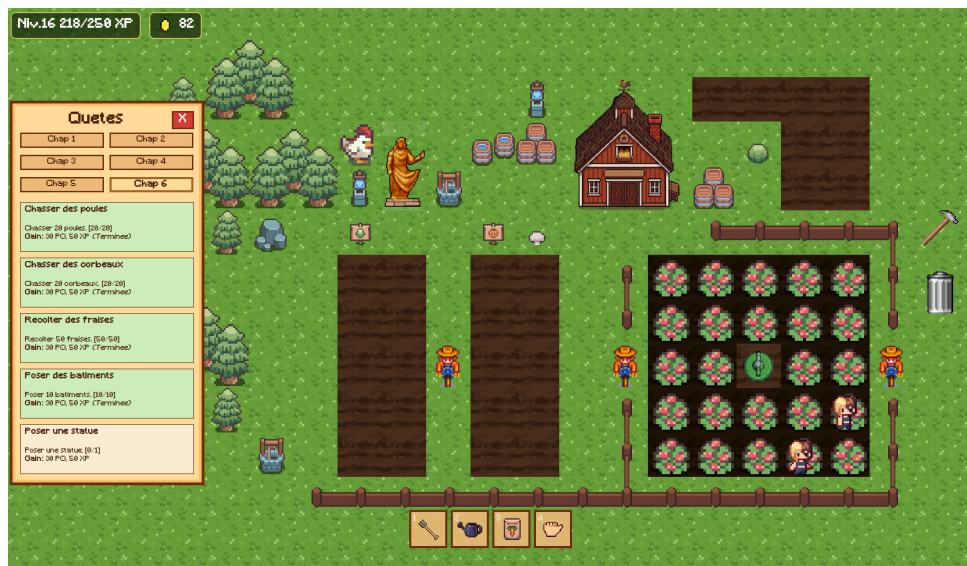


Figure 16 – Panel de quetes complétés

6.3 Une Boucle de Jeu Aboutie

Le produit offre une boucle de progression gratifiante : du premier labourage manuel à l'automatisation complète par arroseurs automatiques, le joueur ressent une réelle montée en puissance. Le système de quêtes chapitrées guide l'utilisateur à travers toutes les fonctionnalités, transformant le simple bac à sable agricole en une véritable aventure scénarisée. La stabilité du système de sauvegarde permet de figer des fermes complexes et de les retrouver à l'identique, témoignant de la fiabilité de l'architecture logicielle mise en place.



Figure 17 – Ecran de gestion des sauvegardes

7 Documentation Utilisateur

Ce manuel d'utilisation a pour vocation d'expliquer le lancement et les contrôles du jeu. Il détaille les prérequis techniques pour lancer le jeu et explique les interactions fondamentales pour développer votre ferme.

7.1 Prérequis Techniques et Lancement

Pour exécuter la simulation, votre ordinateur doit être équipé du Java Development Kit (JDK), idéalement dans sa version 11 ou supérieure, le jeu reposant sur la bibliothèque graphique native Java Swing. L'installation requiert un environnement de développement (comme IntelliJ IDEA ou Eclipse), il vous suffira alors d'importer le projet et d'exécuter la classe Main.

7.2 Premier Démarrage et Gestion des Parties

Dès l'ouverture, vous êtes accueilli par l'écran principal. Cliquez sur le bouton Jouer pour faire apparaître l'interface de gestion des sauvegardes. Cette fenêtre vous permet de saisir le nom d'une nouvelle exploitation au clavier pour créer une nouvelle partie. Si vous avez déjà joué, cette même fenêtre listera vos anciennes parties : cliquez simplement sur le bouton charger de l'une d'elles pour reprendre votre progression exactement là où vous l'aviez laissée. Une fois le nom validé, la partie se lance sur la carte.

7.3 Contrôles et Navigation Visuelle

L'interaction avec votre environnement a été organisée autour d'une hotbar en bas de l'écran. La hotbar se trouve en bas de l'écran. Cette barre contient vos quatre outils fondamentaux. Vous pouvez basculer d'un outil à l'autre très rapidement en pressant les touches numériques 1, 2, 3 et 4 de votre clavier. L'outil actuellement équipé est mis en évidence par un cadre blanc épais.

Pour déplacer votre caméra et explorer l'immensité de votre domaine, il vous suffit d'utiliser les flèches directionnelles ou déplacer le curseur de la souris près des bords de l'écran.. Enfin, la touche Échap de votre clavier est votre meilleur allié : elle agit de manière intelligente pour annuler la pose d'un bâtiment, fermer un menu (comme l'inventaire), ou ouvrir le menu de pause si vous êtes simplement en train d'observer votre ferme.

7.4 De la Terre à la Vente

Votre quotidien s'articule autour du cycle de la vie végétale. Pour commencer, sélectionnez la Houe (outil n°1) et cliquez avec le bouton gauche de la souris sur une case de terre herbeuse pour la labourer. Ensuite, équipez votre Sac de graines (outil n°3) et cliquez sur cette parcelle fraîchement retournée. Un menu apparaîtra pour vous laisser choisir la semence à planter.

Une plante a besoin d'attention pour survivre. Équipez l'Arrosoir (outil n°2) et cliquez sur vos pousses pour maintenir leur niveau d'humidité : le sol deviendra plus foncé pour vous confirmer que l'eau a bien été donnée. Une plante privée d'eau finira par mourir ! (Une plante complètement inondée aussi !) Une fois la plante arrivée à maturité, sélectionnez l'outil de Récolte (outil n°4) et cliquez dessus pour l'arracher et l'ajouter automatiquement à l'inventaire de votre grange. Cet outil sert également à nettoyer les plantes mortes pour libérer le sol.

La Grange, ce grand bâtiment présent sur votre terrain, est le cœur économique de votre ferme. En cliquant dessus, vous ouvrez l'interface marchande. C'est ici que vous pourrez revendre vos récoltes pour amasser de l'argent (PO), puis réinvestir ce capital en achetant de nouvelles graines plus rentables.

Afin de mieux appréhender les mécanismes fondamentaux de la simulation, un Tutoriel est mis à votre disposition.

7.5 Construire et Gérer les Menaces

L'argent récolté vous permet de personnaliser votre ferme. Un panneau de construction est accessible depuis l'interface pour acheter des éléments fonctionnels (Puits, Barrières, etc.). Pour placer un bâtiment, sélectionnez-le : une version translucide du bâtiment suivra votre curseur. S'il est surligné en bleu, un simple clic valide l'achat et la construction. S'il est rouge, c'est que l'emplacement est obstrué ou invalide.

Soyez vigilant, votre exploitation attirera rapidement la convoitise ! À mesure que vous gagnerez des niveaux, des nuisibles comme les poules et les corbeaux tenteront de dévorer vos cultures. Si les clôtures que vous construisez suffisent à bloquer les poules (les obligeant à faire le tour), il vous faudra investir dans des Épouvantails pour repousser les corbeaux qui survolent vos défenses terrestres. Si une poule s'approche trop, vous pouvez également cliquer directement dessus pour l'effrayer et la faire fuir.

7.6 Mettre le Jeu en Pause et Quitter

Vous pouvez suspendre le temps à tout moment en pressant la touche Échap. Depuis cet espace, vous pouvez ajuster les volumes sonores du jeu. C'est également ici que se trouve le bouton Sauvegarder et Quitter. Son utilisation est cruciale : elle garantit que l'intégralité de votre progression est inscrite en sécurité sur votre disque dur avant de vous ramener à l'écran d'accueil.

8 Documentation Développeur : Guide de Reprise et d'Évolution

Ce document s'adresse à tout développeur ou architecte logiciel souhaitant reprendre ce code pour modifier l'équilibrage ou ajouter de nouvelles mécaniques. Le projet reposant sur une architecture MVC stricte et du multithreading, une bonne compréhension des points d'entrée est primordiale.

8.1 Point d'Entrée et Classes Fondamentales

Pour appréhender le projet, il n'est pas nécessaire de lire toutes les classes d'un coup. Voici le cheminement de lecture recommandé :

- **La classe Main (Le Point d'Entrée)** : C'est ici que se trouve la méthode `public static void main(String[] args)`. Elle est très courte et sert uniquement de point de départ pour initialiser la fenêtre (JFrame) et afficher l'écran d'accueil (HomeScreenPanel). C'est également elle qui instancie le Timer global gérant la boucle de rendu.
- **La classe Rendering** : C'est le cœur de l'affichage. Lisez sa méthode `run()` et la méthode `paintComponent(Graphics g)` de la classe Display pour comprendre comment le moteur dessine la grille, la caméra et les entités (le z-index visuel).
- **La classe World** : C'est la base de données vivante de la simulation. Étudiez sa méthode `tick()`, qui est le métronome du jeu, appelant à son tour les `tick()` des plantes et les `applyEffect()` des bâtiments.
- **Le package des Actions (Action, ActionBuilder)** : Pour comprendre comment les agents interagissent avec le monde, regardez l'interface Action et l'implémentation de PlantAction ou MoveAction.

8.2 Principales Constantes et Équilibrage

L'équilibrage (difficulté, temps, économie) n'est pas codé en dur au milieu des algorithmes, mais centralisé via des constantes que vous pouvez modifier instantanément pour changer le comportement du jeu :

Moteur Graphique et Temporel :

- `Rendering.FPS` : Modifie la fluidité visuelle (par défaut 60).
- `World.WIDTH` et `World.HEIGHT` : Dans la génération de la carte, pour agrandir ou réduire la taille de la grille cultivable.

Biologie et Économie (Enum PlantType) :

- Modifiez les attributs de chaque plante pour altérer la difficulté : `growthDuration` (temps avant maturité), `waterConsumption` (vitesse d'assèchement) et `value` (prix

de vente, qui détermine automatiquement le prix d'achat).

Génération des Nuisibles (**ChickenSpawner** / **CrowSpawner**) :

- **SPAWN_INTERVAL** : Détermine la fréquence d'apparition (ex : 20000 ms). Le réduire rendra le jeu beaucoup plus agressif.
- **MAX_CAPACITY** : Le nombre maximal d'entités simultanées pour éviter de surcharger le processeur.

Progression (Stats) :

- L'algorithme dans `getExpForNextLevel()` contient les paliers d'expérience. Modifier ces valeurs accélérera ou ralentira la montée en niveau.

8.3 Pistes d'Amélioration et Implémentations Futures

Plusieurs fonctionnalités de gameplay ont été théorisées mais n'ont pas encore été implémentées. Voici comment vous pourriez les intégrer de manière élégante en respectant l'architecture existante.

8.3.1 Système de Lootbox

Objectif : Offrir au joueur des boîtes mystères pouvant contenir de l'expérience, de l'argent ou débloquent des graines/bâtiments rares. Piste d'implémentation :

- Créez une nouvelle classe `ItemLootbox` héritant de `Item`.
- Dans le `PopupInventory` (ou l'inventaire rapide), ajoutez un écouteur d'événement spécifique : un clic droit (ou double-clic) sur cet item appelle une méthode `openLootbox()`.
- Cette méthode utiliserait un système de probabilités (table de loot avec `Math.random()`) pour instancier un objet.
- Si la récompense est de l'expérience, appelez `stats.addExp()`. Si c'est un objet, injectez-le via `barn.addItem()`. Vous pourriez y coupler l'apparition d'un nouveau `PopupPanel` très visuel affichant l'objet gagné avec un effet sonore dédié.

8.3.2 Mécanique de Compostage et Gestion des Déchets

Objectif : Actuellement, une plante pourrie ou mangée est simplement détruite. L'idée est de valoriser cette perte en permettant sa transformation en engrais. Piste d'implémentation :

- **Création du Bâtiment** : Créez une classe `CompostBuilding` (héritant de `Building`). Donnez-lui un inventaire interne capable de stocker uniquement un nouvel item : `ItemRotten`.

- **Modification de la Récolte** : Modifiez la classe `HarvestAction`. Lors de la méthode `perform()`, si la plante ciblée est dans l'état `POURRIE` ou `MORT`, ne vous contentez pas de faire `plant = null`. Instanciez un `ItemRotten` et ajoutez-le à l'inventaire du jardinier.
- **Pathfinding** : C'est le défi principal. Modifiez la boucle de comportement du `Gardener`. S'il possède un `ItemRotten` dans ses poches, il doit utiliser l'algorithme A^* pour cibler le `CompostBuilding` le plus proche, s'y déplacer, et transférer l'objet (via une nouvelle `DepositAction`).
- **Transformation** : Dans la méthode `applyEffect(World)` du `CompostBuilding` (appelée à chaque tick), implémentez un compteur. Tous les X ticks, si le bâtiment contient des déchets, il les détruit et génère un `ItemFertilizer`, que le joueur pourra récupérer.

9 Conclusion et Perspectives

Bilan des Réalisations

Au terme de ce projet, nous avons développé la conception, l'architecture et l'implémentation complète d'une simulation agricole 2D en temps réel. Nous avons réussi à bâtir un moteur de jeu robuste reposant exclusivement sur la bibliothèque native Java Swing. Le projet combine plusieurs aspects techniques, il s'agit d'une application composée de plusieurs modules intégrant un cycle de croissance des plantes, une intelligence artificielle autonome basée sur l'algorithme A^* , une économie transactionnelle et un système de fenêtrage personnalisé. L'application s'appuie sur une organisation MVC, ce qui sépare la logique, l'affichage et les contrôles.

Difficultés Rencontrées et Solutions Apportées

La réalisation d'un tel projet s'est inévitablement heurtée à des difficultés techniques :

- **La gestion de la concurrence** : Faire cohabiter une boucle de rendu à 60 FPS, une horloge biologique asynchrone et des agents autonomes interagissant avec la même grille a rapidement provoqué des exceptions de type `ConcurrentModificationException`. Nous avons résolu ce problème en instaurant un ordre d'arrêt et de mise en pause des threads et en implémentant des mécanismes d'interruption sécurisés, pour éviter les modifications pendant la sauvegarde ou le rendu.
- **La persistance d'un monde complexe** : La tentative naïve de sérialiser l'objet `World` s'est soldée par des échecs liés aux composants graphiques natifs de Java. Nous avons choisi d'utiliser un DTO nous avons implémenté le patron de conception DTO (Data Transfer Object) couplé à l'API de Réflexion Java, permettant d'extraire uniquement les données et de reconstruire le monde dynamiquement à la lecture.

Acquis et Apprentissages

Ce projet nous a permis de pratiquer Java, Swing et les threads, tant sur le plan technique que méthodologique, en lien avec les notions vues en cours de Programmation Concurrente et Interfaces Interactives.

Au-delà du perfectionnement de notre maîtrise de Java et de l'ingénierie logicielle (MVC, couplage faible, threads), nous avons concrètement expérimenté les enjeux de la gestion de projet. La nécessité de structurer un développement aussi vaste a mis en lumière l'efficacité d'un cycle en V. Le découpage de notre travail en objectifs à court terme et la gestion d'un carnet de tâches (à l'aide d'un diagramme de Gantt) nous ont permis de prioriser un Produit Minimum Viable fonctionnel avant d'y greffer les mécaniques complexes (comme la sauvegarde).

De plus, ce travail a renforcé notre rigueur dans l'utilisation des outils de versionnement collaboratif (Git/GitHub), indispensables pour sécuriser notre base de code. Enfin, nous avons pris pleinement conscience de l'importance cruciale de la phase de documentation. Comme l'illustrent nos sections dédiées, rédiger un manuel utilisateur accessible et un guide développeur détaillé n'est pas une simple formalité académique, mais un point utile pour faciliter la reprise du code pour faciliter la maintenance et la transmission du projet.

Évolutions Futures

Si le cahier des charges initial a été respecté, ce socle technique ouvre la voie à de nombreuses perspectives d'évolution :

- **Refonte du Moteur de Rendu** : Swing a montré ses limites en matière d'optimisation matérielle. Une migration vers un framework plus moderne permettrait d'exploiter l'accélération matérielle pour afficher des cartes plus grandes et effets visuels supplémentaires.
- **Architecture Client/Serveur** : Actuellement fonctionnant en local, le moteur pourrait être scindé pour intégrer des Sockets réseau, transformant la ferme en une expérience multijoueur coopérative.